

Konstantin Lucius Regenhardt

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Umweltinformatik*
im Fachbereich 2
der Fakultät Technik und Informatik
der Hochschule für Technik und Wirtschaft Berlin

Betreuender Prüfer: Prof. Dr. Jochen Wittmann
Zweitgutachter: Ankit Kumar

Eingereicht am: 02.08.2026

Konstantin Lucius Regenhardt

Thema der Arbeit

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Kurzzusammenfassung

Proprietäre Low-Code-Plattformen wie Katalon ermöglichen einen schnellen Einstieg in die Testautomatisierung, können bei wachsender Projektgröße jedoch zu starker Herstellerabhängigkeit und hohen Folgekosten führen. Diese Arbeit entwickelt und evaluiert eine automatisierte Migrationspipeline, die Katalon-Projekte in eine offene Python-Testumgebung mit Selenium und Pytest überführt. Der entwickelte Prototyp verarbeitet Katalon-Testskripte sowie zugehörige Projektartefakte wie Object Repository, Variablen und Konfigurationsdateien und erzeugt daraus eine ausführbare Zielstruktur. In der Evaluation des Beispielprojekts wurden 7 von 10 Testskripten automatisch übersetzt (70%), während 3 Fälle als Restlücke manuell nachbearbeitet werden müssen. Alle Object-Repository-Dateien wurden erfolgreich nach JSON überführt, und die strukturelle Integrität des Zielprojekts wurde durch die fehlerfreie Erkennung der generierten Tests bestätigt. Die Ergebnisse zeigen, dass der Ansatz den manuellen Migrationsaufwand deutlich reduziert und einen praktikablen Ausstiegspfad aus proprietären Testökosystemen bietet, auch wenn komplexe Sonderfälle derzeit noch nicht vollständig automatisiert abgedeckt werden.

Inhaltsverzeichnis

Abkürzungs- und Begriffsverzeichnis	V
Abbildungsverzeichnis	IX
Tabellenverzeichnis	X
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Ziel & Aufbau der Arbeit	2
2 Grundlagen	2
2.1 Automatisiertes Testen	2
2.2 Low-Code-Plattformen im Testbereich	3
2.3 Vendor Lock-in bei proprietären Plattformen	3
2.4 Grundlagen zu Regular Expression (Regex)	4
3 Strukturanalyse	5
3.1 Katalon Projektstruktur	5
3.1.1 Test Cases und testeigene Variablen	5
3.1.2 Object Repository und Test Objects	6
3.1.3 Global Variables und Profile	7
3.1.4 Testdaten und Einbindung	7
3.1.5 Custom Keywords	7
3.2 Verschachtelungen in der Katalon Struktur	8
3.2.1 Verschachtelung der Test Cases und ihrer Variablen	8
3.2.2 Verschachtelung vom Objekt Repository und der Test Objects	9
3.2.3 Verschachtelung der Globalen Variablen	9
3.2.4 Verschachtelung der Testdaten	10
3.2.5 Verschachtelung der Custom Keywords	10
3.3 Python Projektstruktur	11
3.3.1 Testskripte und Variablen	11
3.3.2 Object Repository und Test Objects	12
3.3.3 Testdaten	13
3.3.4 Python Konfiguration	13
4 Konzeption der Migrationspipeline	14
4.1 Hintergrund	14
4.2 Architekturüberblick	14
4.3 Transpilation der Test Cases	15
4.4 Übernahme ergänzender Strukturen	17
4.5 Limitierungen der Migrationspipeline	18
5 Implementierung	18
5.1 Techstack	18
5.1.1 Python	18
5.1.2 Selenium und Pytest	18
5.1.3 Git als Versionierung und GitHub	19
5.2 Projektstruktur des Migrators	19
5.3 Umsetzung der Migrationspipeline	20
5.3.1 Transpilation der Test Cases	21
5.4 Grenzen der Implementierung	27
6 Evaluation	27
6.1 Evaluationskriterien	27

6.2 Migrationsergebnis	27
6.3 Strukturelle Integrität	28
6.4 Codequalität: Vorher-Nachher-Vergleich	30
6.5 Aufwandsvergleich	31
7 Fazit und Ausblick	32
8 Anhang	33
8.1 Regex-Muster der Transpilation	33
8.2 Katalon Projektstruktur	34
Quellenverzeichnis	35
Erklärung zur selbstständigen Bearbeitung	37

Abkürzungs- und Begriffsverzeichnis

API – Application Programming Interface: Eine definierte Schnittstelle, über die Softwarekomponenten miteinander kommunizieren. Selenium stellt eine API bereit, über die Tests den Browser steuern. 2, 3, 22

Assertions: Prüfausdrücke in Testcode, die erwarten, dass eine Bedingung wahr ist. Ist die Bedingung nicht erfüllt, schlägt der Test fehl und markiert die entsprechende Stelle als Fehler. 22

Branches: Parallele Entwicklungszweige in einem Git-Repository, die unabhängige Änderungen am selben Projekt ermöglichen und später zusammengeführt werden können.

Build-Time: Der Prozessablauf während der Ausführung des Migrationswerkzeugs. Im Gegensatz zu Runtime (dem Zeitpunkt der Testausführung) finden alle Transformationen, Konvertierungen und Code-Generierung während der Build-Time statt. In diesem Projekt umfasst die Build-Time alle Pipeline-Module (test_suite_translator, test_transpiler, test_assembler, etc.), die ein Katalon-Projekt einmalig in ein Selenium/Pytest-Projekt konvertieren. 21

CI – Continuous Integration: Ein Softwareentwicklungsprinzip, bei dem Code-Änderungen regelmäßig in ein gemeinsames Repository integriert und automatisch gebaut und getestet werden. Automatisierte Testsuiten sind ein zentrales Werkzeug im CI-Prozess. 19

CLI – Command Line Interface: Eine textbasierte Benutzerschnittstelle, über die Nutzer Programme durch Eingabe von Befehlen steuern. 3

Compiler: Ein Programm, das Quellcode einer Programmiersprache analysiert und auf Fehler prüft oder in eine andere Form überführt. In dieser Arbeit bezeichnet Compiler den statischen Typ-Analysator Pyright, der Python-Dateien vor der Ausführung auf Typ- und Importfehler untersucht.

Delimiter: Ein Trenn- oder Begrenzungszeichen, das den Anfang, das Ende oder die Struktur eines Ausdrucks markiert. Im Kontext von Regular Expressions wird ein Sonderzeichen wie der Punkt durch Escaping (z. B. `\. `) als literales Zeichen behandelt und verliert seine spezielle Regex-Bedeutung. 4, 4

Pytest Fixture: Ein Mechanismus in Pytest, der wiederverwendbare Test-Vorbedingungen und -Nachbedingungen definiert. Fixtures werden in der Datei conftest.py deklariert und stehen allen Tests im Projekt automatisch zur Verfügung, ohne explizit importiert werden zu müssen. Typische Anwendungen sind das Erstellen eines WebDriver-Objekts oder das Einrichten von Testdaten.

Framework: Ein wiederverwendbares Grundgerüst aus Bibliotheken, Regeln und Konventionen, das die Entwicklung einer bestimmten Art von Software strukturiert. In dieser Arbeit dienen Katalon und Selenium/Pytest als Frameworks für die Testautomatisierung.

Git: Ein verteiltes Versionskontrollsystem zur Nachverfolgung von Änderungen an Quellcode und Dateien. Git ermöglicht unter anderem Commit-Historien, parallele Entwicklungszweige und das Zusammenführen von Änderungen.

GitHub: Eine webbasierte Plattform zur Verwaltung von Git-Repositories mit Funktionen für Zusammenarbeit, Pull Requests, Issues und CI/CD-Integrationen. 19

Groovy: Eine dynamisch typisierte Skriptsprache, die auf der JVM läuft und vollständig mit Java interoperabel ist. Katalon Studio verwendet Groovy als Skriptsprache für Testskripte. In dieser Arbeit wird Groovy-Code mittels Regex-Transformation in Python übersetzt. 5, 10, 14

GUID – Globally Unique Identifier: Eine nahezu weltweit eindeutige Kennung, die zur Identifikation von Objekten oder Einträgen verwendet wird. 8, 8

HTML – HyperText Markup Language: Eine standardisierte Auszeichnungssprache zur Strukturierung von Webinhalten. HTML beschreibt den Aufbau von Elementen wie Formularen, Buttons, Tabellen und Textbereichen, auf die automatisierte UI-Tests zugreifen. IX, 6, 6, 21

IDE – Integrated Development Environment: Eine Software-Anwendung, die Entwicklern eine umfassende Umgebung für die Softwareentwicklung bietet. Typischerweise umfasst sie einen Code-Editor, Debugger, Build-Werkzeuge und oft eine grafische Projektansicht. Katalon Studio ist eine solche IDE, die speziell auf Testautomatisierung ausgelegt ist. IX, IX, 5, 5, 6, 7, 8, 8, 8, 10, 14

Interpreter: Ein Programm, das Quellcode zur Laufzeit schrittweise ausführt, ohne ihn vollständig vorab in Maschinencode zu übersetzen. Python wird typischerweise über einen Interpreter ausgeführt. 13

JVM – Java Virtual Machine: Eine Laufzeitumgebung, die plattformunabhängige Ausführung von Java-Bytecode ermöglicht. Groovy, die Skriptsprache von Katalon Studio, läuft auf der JVM. 5

Key-Value-Paare: Eine Datenstruktur, in der Werte einem eindeutigen Schlüssel zugeordnet werden. In dieser Arbeit werden Selektoren und ihre zugehörigen Strategien häufig als Key-Value-Paare beschrieben. 21

Locator: Ein Ausdruck, mit dem Selenium ein HTML-Element auf einer Webseite identifiziert. Gängige Locator-Strategien sind ID, CSS-Selektor, XPath und Name. Katalon speichert Locatoren im Object Repository; bei der Migration werden sie in Selenium-kompatible By-Strategien überführt.

Low-Code: Ein Entwicklungsansatz, bei dem Anwendungen und Prozesse mit stark abstrahierten, meist grafischen Bausteinen erstellt werden. Der manuelle Programmieraufwand ist geringer, dafür sind Erweiterbarkeit und Kontrolle bei komplexeren Anforderungen oft eingeschränkt. 3, 3, 3

Match: Eine erfolgreiche Übereinstimmung eines Regular-Expression-Musters mit einem Teil des Eingabetextes. Wenn ein Regex-Muster einen Textbereich findet, der der definierten Regel entspricht, wird dieser erkannte Bereich als Match bezeichnet. Ein Match enthält typischerweise auch Capture Groups, die Teilinformationen des Musters speichern. 4, 4

Object Repository: Eine zentrale Datenstruktur in Katalon Studio, die alle Test Objects (z. B. Schaltflächen, Eingabefelder) mit ihren Lokalisierungsstrategien speichert. Jedes Objekt enthält Eigenschaften wie XPath, CSS-Selektor oder ID, anhand derer Selenium das Element im DOM der Webanwendung identifiziert. 6, 21

Parsing: Der Prozess, bei dem Text oder Code in eine strukturierte Form überführt wird, sodass Bestandteile wie Klassen, Methoden und Parameter gezielt weiterverarbeitet werden können. IX, IX, 17, 23, 25

PoC – Proof of Concept: Ein Prototyp oder eine Machbarkeitsstudie, die zeigt, dass ein Konzept oder eine Idee praktisch umsetzbar ist. In dieser Arbeit bezeichnet PoC die erste Umsetzung eines Migrationsskripts zur Überprüfung der Machbarkeit. 1

Pro-Code: Ein Entwicklungsansatz, bei dem Lösungen vollständig in einer allgemeinen Programmiersprache umgesetzt werden. Er erfordert mehr Implementierungsaufwand, bietet dafür aber hohe Flexibilität, Transparenz und Anpassbarkeit. 3, 3

Pytest: Ein weit verbreitetes Python-Testframework zur Strukturierung, Ausführung und Auswertung automatisierter Tests. In dieser Arbeit dient Pytest als Test-Runner und organisatorischer Rahmen für die migrierten Selenium-Tests. 1, 3, 11, 18

Python: Eine weit verbreitete, interpretierte Programmiersprache mit klarer Syntax und großem Ökosystem. In dieser Arbeit dient Python als Zielsprache der Migration und als Grundlage für die Ausführung der generierten Selenium/Pytest-Tests. 14

RCP – Rich Client Platform: Ein von Eclipse bereitgestelltes Framework zum Entwickeln modularer Desktop-Anwendungen auf Basis von Plug-ins. 5

Regex – Regular Expression: Ein formaler Ausdruck zur Beschreibung von Zeichenketten-Mustern. In dieser Arbeit werden Regular Expressions zur Transformation von Groovy-Syntax in Python-Syntax eingesetzt. III, 4, 4, 20, 26

Regressionstest: Ein Test zur Überprüfung, ob bereits vorhandene und zuvor funktionierende Software-Funktionalität nach Änderungen weiterhin korrekt arbeitet. Regressionstests werden typischerweise nach Codeanpassungen erneut ausgeführt, um unbeabsichtigte Seiteneffekte frühzeitig zu erkennen.

Repository: Ein versioniertes Archiv für Quellcode und zugehörige Dateien, typischerweise verwaltet mit einem Versionskontrollsystem wie Git. In dieser Arbeit bezeichnet Repository sowohl das Quell-Katalon-Projekt als auch das generierte Ziel-Selenium/Pytest-Projekt als eigenständige, versionierbare Einheiten.

Runtime: Der Zeitpunkt der Ausführung der generierten Tests. Im Gegensatz zu Build-Time (während der Migration) werden während der Runtime die migrierten Selenium-Pytest-Tests tatsächlich ausgeführt. Die in src/runtime/ gespeicherten Dateien (base_test.py, katalon_helpers.py) werden während dieser Phase verwendet und sind nicht Bestandteil des Migrationsprozesses. 22

Selenium: Ein Open-Source-Framework zur Automatisierung von Webbrowsern. In dieser Arbeit wird Selenium in Kombination mit Pytest als Ausführungsumgebung der migrierten Tests verwendet. 1, 2, 3, 4, 11, 18, 21, 22

Spy Web Utility: Ein Werkzeug in Katalon Studio zum Erfassen und Anlegen von Test Objects direkt aus einer laufenden Webanwendung. Dabei werden Selektoren wie XPath, CSS oder Attribute ausgelesen und im Object Repository gespeichert. 6

SUT – System Under Test: Das zu testende System — in dieser Arbeit die Webanwendung, auf die die automatisierten Selenium-Tests angewendet werden.

Test Object: Ein in Katalon Studio definiertes Objekt, das ein HTML-Element auf einer Webseite repräsentiert. Test Objects enthalten Informationen wie Name, Beschreibung und Locator. Sie werden im Object Repository gespeichert und in Testskripten referenziert. 21

- Transpilation:** Der Prozess der automatischen Übersetzung von Quellcode einer Programmiersprache in eine andere. Im Unterschied zur Kompilation bleibt das Abstraktionsniveau erhalten. In dieser Arbeit bezeichnet Transpilation die Übersetzung von Groovy-Testskripten nach Python.
- UI – User Interface:** Die Benutzerschnittstelle einer Anwendung, über die Nutzer mit dem System interagieren. Im Kontext dieser Arbeit bezeichnet UI die grafische Oberfläche, die durch automatisierte Tests validiert wird. 6, 10
- Vendor Lock-in:** Die technische oder wirtschaftliche Abhängigkeit eines Unternehmens von einem einzelnen Anbieter, die einen Wechsel zu Alternativen erschwert oder verteuert. Im Kontext dieser Arbeit bezeichnet Vendor Lock-in die Bindung an Katalon Studio durch proprietäre Dateiformate und lizenzpflichtige Funktionen. III, 3, 3
- WebDriver:** Eine standardisierte API (W3C-Standard), über die Programme einen Webbrowser programmatisch steuern können. Selenium WebDriver ist die Referenzimplementierung und bildet die Grundlage aller in dieser Arbeit generierten Tests. 2, 2
- XML – Extensible Markup Language:** Ein textbasiertes, hierarchisches Datenformat zur strukturierten Beschreibung von Informationen. In dieser Arbeit werden mehrere Katalon-Datentypen als XML interpretiert und in offene Zielformate überführt. 14

Abbildungsverzeichnis

Abbildung 1	Oben: Erst ein Muster zum erkennen von Katalon-Klickaufrufen. Unten: Die Erklärung der einzelnen Bestandteile des Musters (Visualisierung und Validierung mit Regex101[1]).	4
Abbildung 2	Links: Das Dateisystem der Test Cases. Rechts: Ein Test Case mit Testlogik die erst das Fenster „All Users“ öffnet, dann nach dem User „David Kim“ sucht, diesen markiert und danach dessen Details verifiziert.	6
Abbildung 3	Links: Das Dateisystem des Object Repository aufgefaltet. Blau markierte Namen wurden mit Unterstützung der Integrated Development Environment (IDE) eigenen AI generiert. Rechts: Ein Test Object das die Eigenschaften eines HyperText Markup Language (HTML) Elements beschreibt.	6
Abbildung 4	Links: Ansicht der Profile im Dateisystem. Rechts: Ein Profil das die Global Variables URL, USERNAME1 und PASSWORD1 definiert. Die PASSWORD1 Variable ist als „protected“ markiert, sodass ihr Wert in der Integrated Development Environment (IDE) nur maskiert angezeigt wird.	7
Abbildung 5	Links: Ansicht der Testdaten im Dateisystem. Rechts: Ein Testdatensatz aus der Datei users.dat.	7
Abbildung 6	Übersichtsdiagramm der Migrationspipeline von einem Katalon Projekt zu einem ausführbaren Python Projekt. Dargestellt sind die Hauptphasen Initialisierung, Testtranspilation, Übernahme ergänzender Strukturen sowie die Bereitstellung der finalen Projektstruktur mit Inhalten.	15
Abbildung 7	Detailliertes Aktivitätsdiagramm des Transpilationsablaufs eines Katalon-Tests zu einem Python-Test. Dargestellt sind das Einlesen, Filtern, Extrahieren, Parsing, Transformieren, Generieren und Zusammensetzen der Testlogik.	17
Abbildung 8	Durch das katalon_lines_pattern-Muster werden aus einem Block aus Katalon Zeilen, drei getrennte Regex-Matches mit jeweils drei Gruppen für Klasse, Methode und Parameter extrahiert (Visualisierung und Validierung mit Regex101).	24
Abbildung 9	Das fto_as_param_pat-Muster erkennt in der Parsing die findTestObject()-Methode als ersten Parameter, falls ein Komma folgt und ermöglicht es diese zu trennen (Visualisierung und Validierung mit Regex101).	25
Abbildung 10	Das fto_param_str_pattern-Muster extrahiert die findTestObject()-Methode mit dem String-Argument, das für die Transformation benötigt wird (Visualisierung und Validierung mit Regex101).	26
Abbildung 11	Konsolenausschnitt der Migration mit 7 erfolgreichen und 3 fehlgeschlagenen Testübersetzungen.	28
Abbildung 12	Vergleich der Testausführung: oben alle Tests im Katalon-Ausgangsprojekt, unten die Ausführung im generierten Python-Projekt. Die Abweichung entspricht den 3 nicht automatisch übersetzten Fällen.	30
Abbildung 13	Vergleich des Tests filter_for_admins: oben das ursprüngliche Katalon-Skript in Groovy, unten der daraus generierte Python-Test im Zielprojekt.	31

Tabellenverzeichnis

Tabelle 1 Migrationsergebnis 28

Tabelle 2 Regex-Muster die in der Transpilation verwendet werden. 33

1 Einleitung

1.1 Motivation

Automatisiertes Testen ist eine zentrale Komponente moderner Software-Qualitätssicherung. In vielen Teams werden Testautomatisierungs-Frameworks eingesetzt, um Regressionstests zu beschleunigen und das Vertrauen in Veröffentlichungen zu stärken[2]. Anfangs nutzte unser Team Katalon Studio als IDE um neben der Entwicklung einer internen Geschäftssoftware auch eine grundlegende Testinfrastruktur aufzubauen. Im Laufe der Zeit wuchs die Infrastruktur stark an und umspannte den Großteil der Software.

Da sich die Anforderungen des Projekts weiterentwickelten, benötigte das Team fortgeschrittenere Automatisierungstechniken, wie automatisiertes Berechnen von Datumseinträgen und eine tiefere Kontrolle über die Testausführung und das Debugging. In dieser Phase traten erste Blockaden auf: Erweiterte Funktionen, die im Programmieren als elementar gesehen werden, erforderten zusätzliche Lizenzen, was erhebliche Kosten verursachte und die Skalierung der Infrastruktur einschränkte. Dies kristallisierte sich schnell als ein blockierendes Problem heraus da die Preise in keinem Verhältnis zu den Leistungen standen und über längere Zeit nicht tragbar waren. Als Lösungsansatz wurde eine Migration zu einem Open-Source Projekt vorgeschlagen.

Angesichts der erheblichen bestehenden Investitionen in eine Katalon-basierte Testumgebung war ein vollständiges Neuschreiben aller gesammelten Tests von Grund auf, nicht machbar. Deswegen wurde ein Migrationsansatz als Proof of Concept (PoC) untersucht, um die zuvor erstellten Tests und alles Zugehörige zu erhalten und gleichzeitig den Übergang zu einer offenen und erweiterbaren Open-Source Struktur zu ermöglichen.

1.2 Problemstellung

Das Kernproblem das in dieser Arbeit behandelt wird, ist die Frage wie ein bestehendes Katalon Projekt beim Wechsel in eine lizenzunabhängige Open-Source Umgebung beibehalten und wiederverwendet werden kann. Eine manuelle Migration von Katalon Tests zu Selenium und Pytest Tests ist relativ zeitaufwendig und bei einer großen Testmenge wird dies praktisch unerreichbar. Das liegt daran, dass neben der reinen Skriptübersetzung auch noch eine Menge an zusätzlichen Abhängigkeiten und Konfigurationen berücksichtigt werden müssen. Ein Katalon Projekt besteht nicht nur aus Testskripten, sondern auch aus einer Vielzahl von Teilen wie dem „Objekt-Repository“, testexternen Variablen-Definitionen, Profilen und Datenabhängigkeiten. Diese sind jeweils in proprietären Formaten gespeichert, in ihrer Funktionsweise verschleiert und müssten manuell „ausgegraben“ und zusammengeführt werden. Eine manuelle Migration würde daher nicht nur die Übersetzung von Groovy nach Python erfordern, sondern auch die Rekonstruktion aller Teile der Projektstruktur und ihre Anpassung an die Open-Source-Frameworks.

Die technische Herausforderung liegt nicht nur in der syntaktischen Konvertierung des Testcodes, sondern in der End-to-End-Transformation eines kompletten Testautomatisierungsprojekts in ein ausführbares und skalierbares alternatives Format.

1.3 Ziel & Aufbau der Arbeit

Das Ziel dieser Arbeit ist es, einen automatisierten Migrator zu entwerfen, zu implementieren und zu evaluieren, der Katalon-basierte Testprojekte in eine ausführbare Python-Projektstruktur überführt. Um dieses Ziel zu erreichen wird in dieser Arbeit zuerst die Struktur eines Katalon Projektes und die eines Python Projektes analysiert. Dann wird Stück für Stück ein Migrationsalgorithmus entworfen, der die proprietären Formate des einen, in offen nutzbare Formate des anderen transformiert. Anschließend wird ein Prototyp implementiert, der die Migration automatisiert und die Testlogik so weit wie möglich erhält. Die Evaluation konzentriert sich dabei auf vier Kernfragen: Wie viel des Quellprojekts wird automatisch übersetzt, wie hoch ist der tatsächliche Automatisierungsgrad, in welchem Umfang ist das resultierende Python-Projekt funktional äquivalent zum Katalon-Ausgangsprojekt und welche Fälle bleiben unübersetzt und erfordern manuelle Nacharbeit.

2 Grundlagen

2.1 Automatisiertes Testen

Softwaretests sind ein grundlegender Bestandteil der Qualitätssicherung in der Softwareentwicklung. Ihr Ziel ist es, sicherzustellen, dass ein System korrekt funktioniert und definierte Anforderungen erfüllt. Dabei unterscheidet man grob zwischen manuellen und automatisierten Tests.

Bei manuellen Tests, führt eine Person die Testschritte selbst aus und bewertet das Ergebnis[3]. Dieser Ansatz ist bei kleinen, seltenen oder exemplarischen Überprüfungen gut nutzbar, jedoch skaliert dessen Zeitaufwand schnell. Mit wachsender Softwarekomplexität steigt der Zeitaufwand für manuelle Tests bei vollständiger Abdeckung stark an, da die Anzahl möglicher Kombinationen mit der Anzahl von Eingabeparametern exponentiell wächst[4, S. 34, S. 37].

Automatisiertes Testen bezeichnet das Ausführen von Tests mit Hilfe von Testskripten, die von einem Computer ausgeführt werden. Die Skripte definieren Eingaben, Aktionen und erwartete Ergebnisse. Einer der größten Vorteile davon gegenüber manuellen Tests ist die Frequenz. Man kann sie beliebig oft und sehr konsistent durchführen. Die sich dadurch ergebende Geschwindigkeit erlaubt es, große Mengen an Tests effizient in Form von Test-Suites abzuarbeiten. Nach Codeänderungen kann dadurch zum Beispiel schnell überprüft werden, ob bestehende Funktionalität noch korrekt arbeitet. Dazu kann man automatisierte Tests direkt in CI/CD-Pipelines einbinden und so in den regulären Entwicklungsprozess integrieren[5, S. 884].

Für webbasierte Anwendungen hat sich dabei der Selenium WebDriver als weit verbreitetes Werkzeug etabliert. Der WebDriver ist ein standardisiertes Application Programming Interface (API), über das die Testskripte einen Browser programmatisch steuern. Klicks, Formularein-

gaben und Navigationsbefehle werden direkt an den Browser gesendet, als ob ein Nutzer sie manuell ausführen würde[6].

2.2 Low-Code-Plattformen im Testbereich

Low-Code-Plattformen sind Entwicklungsumgebungen, die komplexe technische Operationen durch grafische Oberflächen und vorgefertigte Bausteine abstrahieren. Im Bereich der Testautomatisierung bieten sie einen niedrighschwelligen Einstieg: Tests können über eine GUI aufgezeichnet oder aus einem Katalog von vordefinierten Aktionen zusammengestellt werden, ohne tiefgehende Programmierkenntnisse vorauszusetzen.

Diese Eigenschaft macht Low-Code-Tools attraktiv für Teams die wenig Programmierkenntnisse haben. Mit wachsender Anforderungskomplexität stoßen auch sie oft schnell an Grenzen[7]. Individuelle Logik, die über vorgefertigte Bausteine hinausgeht, ist dabei schwer oder gar nicht umsetzbar. Da die Testlogik eng an die Plattform gebunden ist, lässt sie sich zudem schlecht in externe Versionskontrollsysteme integrieren. Skalierbarkeit und Anpassbarkeit bleiben durch das Plattformmodell begrenzt.

Im Gegensatz dazu stehen sogenannte Pro-Code-Ansätze, bei denen Tests vollständig in einer allgemeinen Programmiersprache wie Python geschrieben werden. Frameworks wie Selenium und Pytest bieten dabei maximale Flexibilität, erfordern aber entsprechende Programmierkenntnisse. Dabei sind diese Frameworks oft Open-Source, also kostenlos, öffentlich zugänglich und transparent. Der Übergang von Low-Code zu Pro-Code ist inhaltlich der Kern dieser Arbeit.

2.3 Vendor Lock-in bei proprietären Plattformen

Vendor Lock-in beschreibt die Abhängigkeit eines Nutzers oder einer Organisation von einem bestimmten Anbieter, sodass ein Wechsel zu einer Alternative mit erheblichem Aufwand oder hohen Kosten verbunden ist[8]. Diese Abhängigkeit entsteht häufig durch proprietäre Dateiformate, plattformspezifische Programmiersprachen oder API, die außerhalb des jeweiligen Ökosystems nicht verwendet werden können.

Im Bereich der Testautomatisierung zeigt sich Vendor Lock-in beispielsweise dann, wenn:

- Testskripte in einer plattformeigenen Skriptsprache verfasst sind und nicht ohne Weiteres auf andere Frameworks übertragen werden können,
- interne Datenstrukturen wie Objekt-Repositorys oder Profil-Konfigurationen in proprietären Formaten gespeichert sind,
- bestimmte Funktionen nur in kostenpflichtigen Lizenzstufen verfügbar sind.

Die Konsequenz ist, dass Teams entweder rasant steigende Lizenzkosten akzeptieren müssen oder bei einem Plattformwechsel einen erheblichen Teil ihrer bisherigen Arbeit verlieren. Für das in dieser Arbeit beschriebene Projekt manifestierte sich dieses Problem in Form von grundlegenden Entwicklungsfunktionen, wie das Debugging über die Command Line Interface (CLI),

die teurere Lizenzpakete erforderten, was wiederum die langfristige Nutzung der Plattform sehr teuer machte.

2.4 Grundlagen zu Regular Expression (Regex)

Der Migrator verwendet Regular Expression (Regex), um wiederkehrende Muster im Groovy-Quelltext automatisiert zu erkennen. Nach Friedl sind Regular Expressions eine formale Sprache zur Beschreibung von Textmustern[9]. Das bedeutet ein Muster wird definiert, auf Text angewendet und liefert bei Übereinstimmung ein Match.

Hier in Abbildung 1 zum Beispiel das Erkennen von Katalon-Klick:

Regular Expression

```
: / WebUI\\.click\\((.*)\\)
```

Test String

```
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
```

Explanation

- `\(` matches the character `(` with numeric value `4010 (508 or 2816)` literally
- 1st Capturing Group `(.*)`
 - `.*` matches any character (except for line terminators)
 - `*` matches the previous token zero or more times, as many times as possible, giving characters back as needed (greedy)
- `\)` matches the character `)` with numeric value `4110 (518 or 2916)` literally
- Global pattern flags
 - `g` modifier: global. Finds all matches instead of stopping after the first
 - `m` modifier: multiline. Causes `^` and `$` to match the start and end of each line, not only the start and end of the string

Match Information

Match 1	0-59	WebUI.click(findTestObject('All_Users/view_all_users_btn'))
Group 1	12-58	findTestObject('All_Users/view_all_users_btn')

Abbildung 1: Oben: Erst ein Muster zum erkennen von Katalon-Klickaufrufen. Unten: Die Erklärung der einzelnen Bestandteile des Musters (Visualisierung und Validierung mit Regex101[1]).

Das Muster „WebUI\\.click\\((.*)\\)“ findet alle Zeilen die „WebUI.click(...)“ enthalten, erstellt ein Match für jede Übereinstimmung und extrahiert den Inhalt der Klammern als Gruppe. Dieser extrahierte Teil kann anschließend in einen Selenium Methodenaufwurf eingeführt werden. Spezifisch wird zuerst nach der Buchstabenreihenfolge „WebUI.click(„ gesucht. Dabei wird ein Backslash-Delimiter „\\“ vor dem Punkt „.“, genutzt damit dieser seine eigentliche Funktion in der Regex-Sprache verliert und nur als Punkt interpretiert wird. Der gleiche Delimiter kommt

danach für die Klammern ins Spiel. Darauf folgt direkt noch eine Klammer ohne Delimiter welche die Eröffnung einer Gruppe signalisiert. In der Gruppe wird nach einem Punkt „.“ gesucht, das heißt ein Vorkommen eines beliebigen Zeichens außer Satzenden. Der Stern „*“ kopiert die Funktion des vorherigen Zeichens kein mal oder beliebig oft. Dann schließt sich zuerst die Gruppe durch eine Klammer „)“ und dann endet das Muster mit einer delimitierten schließenden Klammer „)“. In der eigentlichen Pipeline werden dafür mehrere Muster kombiniert um alles Geschriebene in Tests wie Klassen, Methoden und Parameter systematisch zu zerlegen. Für die Entwicklung und Visualisierung dieser Muster wurde Regex101 genutzt[1]. Die dort iterativ verfeinerten Ausdrücke wurden anschließend in Python überprüft, damit sie mit der tatsächlichen Laufzeitumgebung des Migrators konsistent bleiben.

3 Strukturanalyse

3.1 Katalon Projektstruktur

Katalon Studio ist eine proprietäre Testautomatisierungsplattform, die technisch auf der Eclipse Rich Client Platform (RCP) aufbaut[10]. Eclipse ist eine klassische Integrated Development Environment (IDE) für Java-Entwicklung. Ein Katalon Studio Projekt besteht, aus dem Blickwinkel des Benutzers aus „Test Cases“, einem „Object Repository“, „Global Variables“ in Profilen, testeigenen Variablen und eingebundenen Testdaten. Diese werden in einer hierarchischen Ordnerstruktur gespeichert, die komplett von der IDE verwaltet wird. Hier werden die wichtigsten Ordner und Dateien eines Katalon-Projekts beschrieben und alle die für dieses Projekt relevant sind.

3.1.1 Test Cases und testeigene Variablen

Die Testskripte in Katalon sind in Groovy geschrieben, einer dynamisch typisierten Sprache, die auf der Java Virtual Machine (JVM) läuft. Sie werden mit einer Vielzahl von eingebauten Keywords und Funktionen geliefert, die speziell für Testautomatisierung entwickelt wurden. Variablen können sowohl global als auch lokal definiert werden, wobei globale Variablen in Profilen gespeichert sind und in allen Tests zugänglich sind. Testeigene Variablen werden im jeweiligen Test definiert und sind nur innerhalb dieses Tests verfügbar. Die folgende Abbildung 2 zeigt Ausschnitte darüber aus der Katalon IDE.

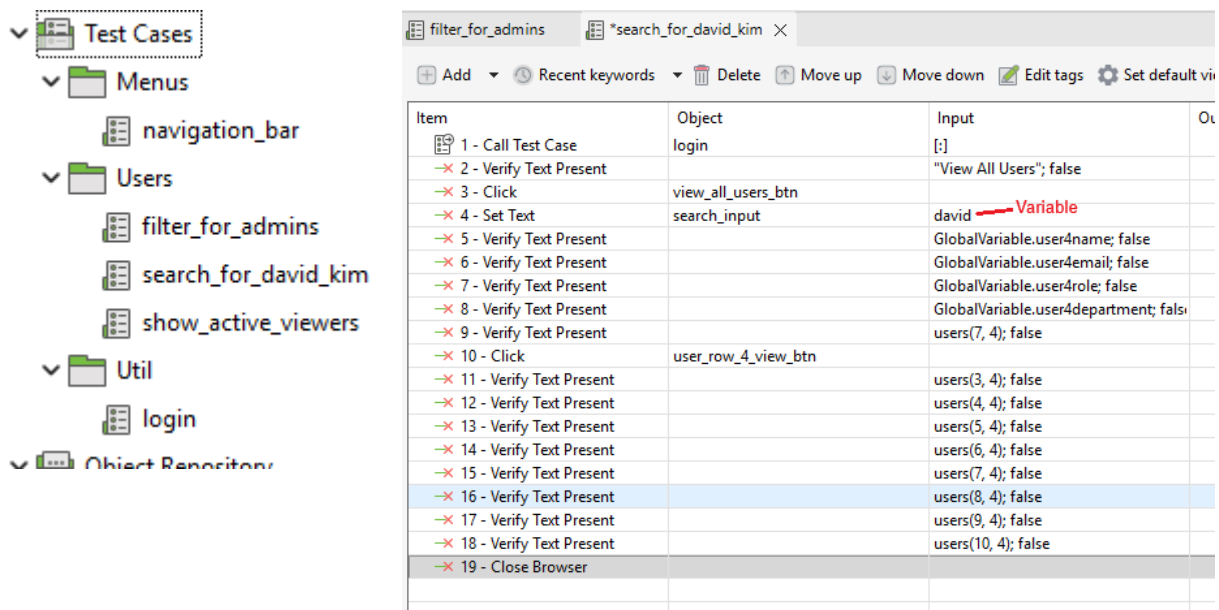


Abbildung 2: Links: Das Dateisystem der Test Cases. Rechts: Ein Test Case mit Testlogik die erst das Fenster „All Users“ öffnet, dann nach dem User „David Kim“ sucht, diesen markiert und danach dessen Details verifiziert.

3.1.2 Object Repository und Test Objects

Das Object Repository ist ein zentrales Element in der Katalon Umgebung, das das Speichern von „User Interface (UI)“-Elementen ermöglicht. Es speichert die Eigenschaften von HyperText Markup Language (HTML) Elementen, die während der Testausführung verwendet werden. Dafür können die User die Elemente mit der Spy Web Utility, die Katalon Studio bereitstellt, auf der Webseite auswählen und im Object Repository generieren lassen[11]. Diese „Test Objects“ können in verschiedenen Tests wiederverwendet werden. Hier eine Abbildung 3 dazu:

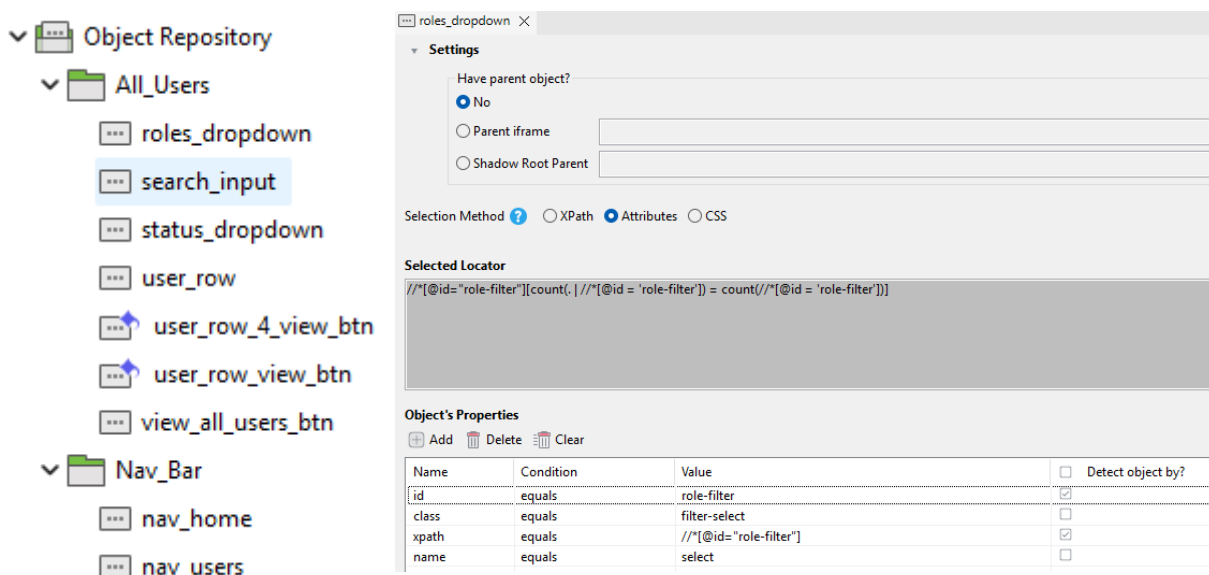


Abbildung 3: Links: Das Dateisystem des Object Repository aufgeklappt. Blau markierte Namen wurden mit Unterstützung der Integrated Development Environment (IDE) eigenen AI generiert. Rechts: Ein Test Object das die Eigenschaften eines HyperText Markup Language (HTML) Elements beschreibt.

3.1.3 Global Variables und Profile

User können in Katalon eigene globale Variablen definieren, die in allen Tests verwendet werden können. Diese Variablen werden in Profilen gespeichert, welche unterschiedliche User oder Umgebungen repräsentieren können. Profile ermöglichen es, Tests in verschiedenen Konfigurationen auszuführen, ohne den Testcode selbst ändern zu müssen. Hat ein Profil zum Beispiel die Variable „URL“ mit dem Wert `https://staging.example.com` und ein anderes Profil die gleiche Variable mit dem Wert `https://production.example.com`, kann ein Test, der die Variable nutzt in beiden Umgebungen ausgeführt werden. Dafür muss nur das entsprechende Profil ausgewählt wird, was die unten stehende Abbildung 4 verdeutlicht.

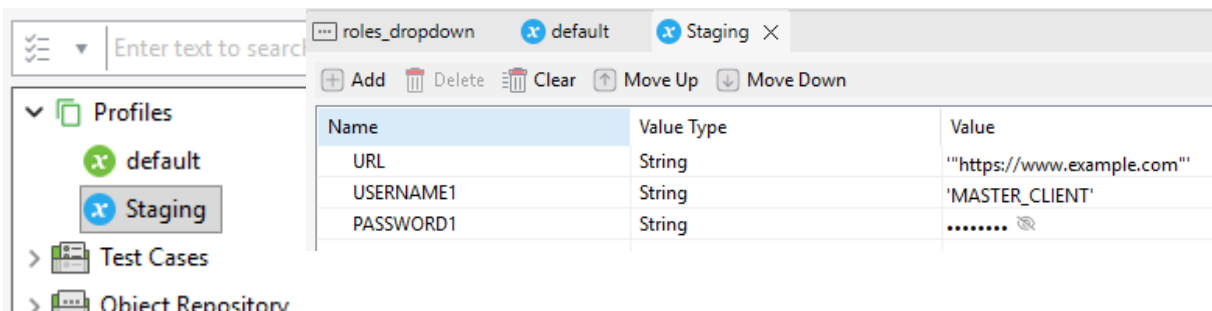


Abbildung 4: Links: Ansicht der Profile im Dateisystem. Rechts: Ein Profil das die Global Variables URL, USERNAME1 und PASSWORD1 definiert. Die PASSWORD1 Variable ist als „protected“ markiert, sodass ihr Wert in der Integrated Development Environment (IDE) nur maskiert angezeigt wird.

3.1.4 Testdaten und Einbindung

Katalon kann Testdaten aus verschiedenen Quellen, wie Excel-Dateien, CSV-Dateien oder Datenbanken einbinden. Diese Testdaten können in den Testskripten referenziert werden, um die Tests mit unterschiedlichen Eingabewerten auszuführen. Abbildung 5 zeigt einen Ausschnitt der Dateisystems:

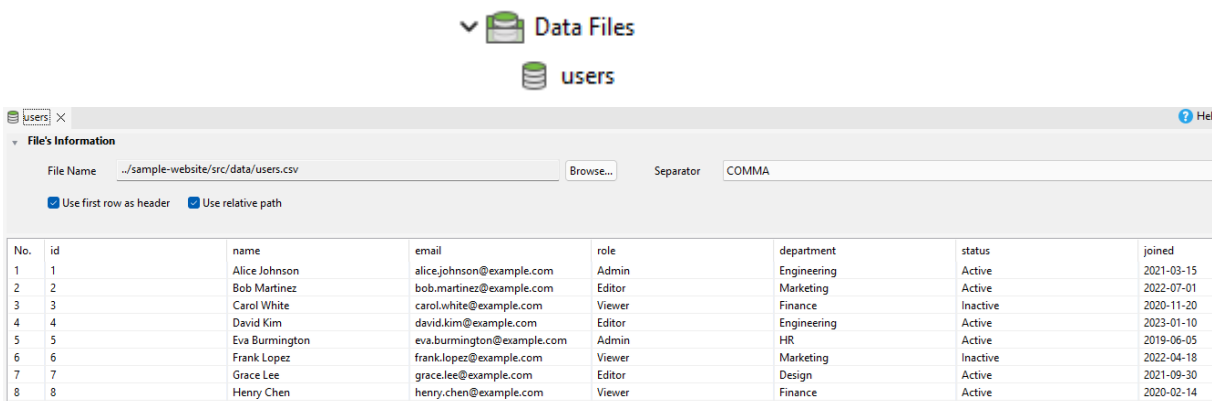


Abbildung 5: Links: Ansicht der Testdaten im Dateisystem. Rechts: Ein Testdatensatz aus der Datei users.dat.

3.1.5 Custom Keywords

User können eigene Strukturen, sogenannte „Custom Keywords“ erstellen, die in mehreren Tests wiederverwendet werden können. Sie funktionieren wie selbst geschriebene Methoden

in klassischen objekt-orientierten Programmiersprachen. Diese Methoden werden in Groovy geschrieben und ermöglichen es, komplexere Logik für Tests zu nutzen und diese wiederverwendbar zu machen.

3.2 Verschachtelungen in der Katalon Struktur

Alle Dateien und Ordner die man in der Katalon IDE erstellt, werden durch den Viewport übersichtlich dargestellt. Sobald man sich die Ordnerstruktur eines Katalon Projekts jedoch außerhalb der IDE anschaut, erkennt man, dass die jegliche selbsterstellte Inhalte nicht nur jeweils eine einzelne Datei sind, wie im Programmieren üblich, sondern aus mehreren Dateien bestehen.

3.2.1 Verschachtelung der Test Cases und ihrer Variablen

Ein „Test Case“ besteht in Katalon aus mehreren Dateien, die zusammenarbeiten. Folgt man der Katalon-Ordnerstruktur im Explorer unter „Test Cases“, so wie sie in der IDE zu sehen ist, landet man bei einer .tc-Datei. Öffnet man diese, erkennt man schnell, dass es sich dabei um eine umbenannte .xml-Datei handelt. Sie enthält Meta-Daten, wie eine Beschreibung, den Testnamen, Tags, Kommentare, eine Globally Unique Identifier (GUID) und die eigentlichen Werte der testeigenen Variablen. Es gibt keine offensichtliche Referenz auf die eigentliche Testlogik. Diese ist in einer weiteren Datei gespeichert. Man würde annehmen die GUID hilft bei der Referenzierung der beiden Dateien, doch die Testskripte enthalten diese an keiner Stelle. Um sie zu finden muss man den Ordner „Scripts“ öffnen, dann den Pfad des „Test Cases“ spiegeln. Dort befindet sich ein Ordner, der den Testnamen trägt und dieser enthält dann eine .groovy-Datei, die den Namen „Script“ kombiniert mit einer zufälligen Nummer trägt. Die Nummer steht in keiner Verbindung zu der .tc-Datei. In dieser .groovy-Datei befindet sich dann die eigentliche Testlogik. Greift der Test auf eine Variable zu, wird im Skript der „rohe“ Name der Variable ausgeschrieben. „Roh“ bedeutet, dass der Name im Code keinen Datentyp hat und auf nichts im Test verweist. Die Variable wird im Test weder deklariert, noch initialisiert. Listing 1 verdeutlicht die Ordnerstruktur der „Test Cases“.

```

sample-website-katalon-tests/
|
├─ Test Cases/
|   └─ Users/
|       └─ filter_for_admins.tc
|       └─ search_for_david_kim.tc
|       └─ show_active_viewers.tc
|   └─ Menus/
|       └─ navigation_bar.tc
|
└─ Scripts/
    └─ Users/
        └─ filter_for_admins/
            └─ Script1781348815820.groovy
        └─ search_for_david_kim/
            └─ Script1781447000826.groovy
        └─ show_active_viewers/
            └─ Script...groovy
    └─ Menus/
        └─ navigation_bar/
            └─ Script...groovy

```

Listing 1: Die Abbildung zeigt die Ordnerstruktur der Test Cases und der zugehörigen Testskripte aus der Perspektive des Dateisystems. Die .tc-Dateien enthalten Meta-Daten und die .groovy-Dateien die eigentliche Testlogik.

3.2.2 Verschachtelung vom Objekt Repository und der Test Objects

Will ein Test ein „Test Object“ verwenden, wird im Testskript der komplette Pfad zum Speicherort der .rs-Datei angegeben um diese aufzurufen. Die .rs-Datei enthält alle Selektoren und die Angabe welcher davon als Hauptselektor verwendet werden soll. Eine Übersicht der Ordnerstruktur des „Object Repository“ und der Test Objects ist in Listing 2 zu sehen.

```

sample-website-katalon-tests/
|
└─ Object Repository/
    └─ All_Users/
        └─ view_all_users_btn.rs
        └─ search_input.rs
        └─ roles_dropdown.rs
        └─ status_dropdown.rs
        └─ user_row.rs
        └─ user_row_view_btn.rs
    └─ Nav_Bar/
        └─ ...weitere Test Objects

```

Listing 2: Die Abbildung zeigt das Objekt Repository und die Test Objects aus der Perspektive des Dateisystems. Die .rs-Dateien enthalten die Selektoren von HTML-Elementen.

3.2.3 Verschachtelung der Globalen Variablen

Wenn der Test eine globale Variable verwendet, zeigt sich dies im Code als „GlobalVariable.VARIABLENNAME“. Der Wert der Variable ist nicht im Skript zu finden, sondern in der Profil-Datei mit der Endung .gbl, die im Ordner „Profiles“ gespeichert ist. Diese Datei

ist ebenfalls eine .xml-Datei und enthält die Variablen als Eintrag in einer „GlobalVariableEntity“ - ein Eintrag der die Variable beschreibt. Diese Einträge enthalten den tatsächlichen Wert und Metadaten, bestehend aus einer Beschreibung, dem Variablennamen, dem Datentyp und einem Boolean der beschreibt ob der Wert „protected“ ist. Ist er „protected“, wird der Wert als sensibel behandelt und in der UI und bei der Bearbeitung nur mit dem Stern-Charakter maskiert angezeigt. In „Logs“ werden die Werte dann auch nicht angezeigt. In der .gbl-Datei ist der Wert jedoch unverschlüsselt und im Klartext zu sehen. Je nach dem welches Profil in der IDE ausgewählt ist, wird die entsprechende .gbl-Datei geladen und die Variablenwerte werden im Test verwendet. Um das zu verdeutlichen sieht man in Listing 3 die Ordnerstruktur der Profile und die .gbl-Dateien.

```
sample-website-katalon-tests/  
|  
└─ Profiles/  
    └─ default.glbl  
    └─ staging.glbl
```

Listing 3: Die Abbildung zeigt die Ordnerstruktur der Profile aus der Perspektive des Dateisystems. Die .gbl-Dateien enthalten die Global Variables und deren Werte.

3.2.4 Verschachtelung der Testdaten

Sobald man mit Katalon Studio ein Dateiformat mit Daten einbinden möchte, erstellt Katalon Studio eine .dat-Datei im Ordner „Data Files“ die den gleichen Namen wie die eingebundene Datei trägt. Die Datei enthält ebenfalls XML-Strukturen, die Metadaten enthalten. Unter anderem den Pfad zur eingebundenen Datei, die Dateiart, ggf. die Trennzeichen und ob der Pfad projektintern oder -extern ist. Die eigentlichen Daten sind nicht in der .dat-Datei enthalten, sondern in der eingebundenen Datei. In Listing 4 ist die Ordnerstruktur der Testdaten zu sehen.

```
sample-website-katalon-tests/  
|  
└─ Data Files/  
    └─ users.dat
```

Listing 4: Die Abbildung zeigt die Ordnerstruktur der Testdaten aus der Perspektive des Dateisystems. Die .dat-Dateien enthalten Meta-Daten und verweisen auf die eigentlichen Testdaten.

3.2.5 Verschachtelung der Custom Keywords

Der Großteil der „Custom Keywords“-Logik wird im Ordner „Keywords“ und selbst erstellten Unterordnern gespeichert, und ist in Groovy in der Form „KlassenName.groovy“ geschrieben. Diese Skripte enthalten die eigentlichen Klassen und Methoden die in den Tests aufgerufen werden. Im Ordner „Libs“ gibt es die „CustomKeywords.groovy“-Datei. Diese definiert statische Weiterleitungen mit dem gleichen Namen wie die Klassen im Ordner „Keywords“. Diese Methoden rufen die eigentlichen Methoden in den Klassen auf und machen sie dadurch in jedem Test aufrufbar. Die Ordner in denen Keywords gespeichert werden, werden in Listing 5 gezeigt.

```
sample-website-katalon-tests/  
|  
├─ Keywords/  
|   └─ data/  
|       └─ TESTER.groovy  
|  
└─ Libs/  
    └─ CustomKeywords.groovy
```

Listing 5: Die Abbildung zeigt die Ordnerstruktur der Custom Keywords aus der Perspektive des Dateisystems. Die .groovy-Dateien im Keywords-Ordner enthalten die Logik der Klassen und Methoden, die in den Tests aufgerufen werden. Im Libs-Ordner befindet sich die CustomKeywords.groovy-Datei, die statische Weiterleitungen zu den Klassen im Keywords-Ordner definiert.

3.3 Python Projektstruktur

Ein Projekt, das Selenium und Pytest für die Testautomatisierung nutzt, ist im Gegensatz zu Katalon nicht an eine proprietäre Projektstruktur gebunden. Pytest erwartet nur eine nachvollziehbare Dateioorganisation durch Konventionen, gültige Python-Module und eine Konfiguration im Projektwurzelverzeichnis, der „root“, in Form einer pyproject.toml oder pytest.ini[12]. In einem reinen Testprojekt enthält das Repository dabei keinen Anwendungscode, sondern ausschließlich Tests, gemeinsam genutzte Hilfsmodule, Konfigurationsdateien und Testdaten. Typischerweise existiert ein zentrales „tests“-Verzeichnis, das bei wachsender Projektgröße weiter in Teilbereiche der zu testenden Anwendung gegliedert wird, beispielsweise in UI-Tests und API-Tests[13]. Das aus Python-Paketprojekten bekannte src-Layout ist in diesem Fall nicht zwingend erforderlich. Laut Python Packaging User Guide dient das src-Layout vor allem dazu, importierbaren Anwendungscode klar von der Projekt-root zu trennen[14]. Da dieses Python Projekt in Abhängigkeit zur Katalon Struktur steht, wird die ursprüngliche Katalon Struktur in der Python Struktur wiedergespiegelt. Das src-Layout wird genutzt um die Katalon Strukturen von den üblichen Python Strukturen zu trennen.

3.3.1 Testskripte und Variablen

Die eigentlichen Testfälle folgen den von Pytest vorgesehenen Namenskonventionen wie test_NAME.py oder NAME_test.py[13]. Anders als bei Katalon besteht ein Testfall dabei in der Regel aus genau einer Python-Datei, in der die Testlogik direkt lesbar ist. Wiederverwendbare Element-Selektoren oder Methoden können in separate Hilfsmodule ausgelagert werden. Diese heißen oft „Helper“- oder „Utility“-Funktionen.

Variablen werden in einem solchen Projekt normalerweise nicht über proprietäre Profil- oder Metadateien verwaltet, sondern über normale Python-Programmierstrukturen. Typisch sind Variablen direkt in einem Testskript oder werden in einer zentralen Konfigurationsdatei definiert. Dadurch bleibt nachvollziehbar, woher ein Wert stammt und an welcher Stelle er in den Test eingebunden wird.

Da Katalon Studio die Variablen in eigenen Dateien speichert und sie mithilfe eines Pfades aufruft, muss der Migrator die gleiche Struktur erstellen, um die Tests unverändert ausführen zu können. Es wird der Ordner „variables“, wie in Listing 6 zu sehen, angelegt der die Struktur der Test Cases mit Variablen widerspiegelt. In den Unterordnern werden die Variablen in Python-Klassen gespeichert, die jeweils alle Variablen des Tests enthalten. Die Klassen werden dann in den Tests importiert und die Variablen können aufgerufen werden.

```

sample-website-selenium-tests/
├──
└── src/
    ├── __init__.py
    ├── tests/
    │   ├── Users/
    │   │   ├── test_filter_for_admins.py
    │   │   ├── test_search_for_david_kim.py
    │   │   ├── test_show_active_viewers.py
    │   │   └── __init__.py
    │   ├── Menus/
    │   │   ├── test_navigation_bar.py
    │   │   └── __init__.py
    │   └── __init__.py
    └── variables/
        ├── Users/
        │   ├── var_search_for_david_kim.py
        │   └── __init__.py
        └── __init__.py

```

Listing 6: Die Abbildung zeigt die Struktur der Testskripte und der testeigenen Variablen im Zielprojekt. Die Tests sind in Python-Dateien organisiert, die den Testlogik enthalten. Die Variablen sind in separaten Python-Klassen gespeichert, die jeweils alle Variablen eines Tests enthalten.

3.3.2 Object Repository und Test Objects

So wie Testskripte und Variablen in der Python Struktur gespiegelt werden, werden auch das Object Repository und die Test Objects darin gespiegelt. Die Test Objects werden in JSON-Dateien gespeichert, die die gleichen Informationen enthalten wie die ursprünglichen .rs-Dateien. Die Struktur der Quellordner und Unterordner wird beibehalten, sodass die Tests weiterhin auf die Test Objects zugreifen können, indem sie den Pfad zu den JSON-Dateien angeben. Zu sehen ist das in Listing 7.

```
sample-website-selenium-tests/  
|  
└─ src/  
    └─ __init__.py  
        └─ object_repository/  
            └─ All_Users/  
                └─ view_all_users_btn.json  
                └─ search_input.json  
                └─ roles_dropdown.json  
                └─ __init__.py  
            └─ Nav_Bar/  
                └─ __init__.py
```

Listing 7: Die Abbildung zeigt die Struktur des Object Repositories und der „Test Objects“-Dateien die darin im JSON-Format gespeichert sind.

3.3.3 Testdaten

Testdaten werden häufig in einem eigenen Verzeichnis wie z. B. „data“, „resources“ oder „testdata“ abgelegt und dann bei Bedarf direkt von den Tests oder von Hilfsmodulen eingelesen. Dabei kann es sich beispielsweise um JSON-, CSV-, XML- oder Excel-Dateien handeln, jedoch ist praktisch jedes Format nutzbar. In diesem Fall wird ein „data“-Ordner angelegt, der die Testdaten aus Katalon direkt kopiert und in Listing 8 zu sehen ist. Die Tests können dann auf die Daten zugreifen, indem sie den Pfad zu den Dateien angeben.

```
sample-website-selenium-tests/  
|  
└─ data/  
    └─ users.csv
```

Listing 8: Die Abbildung zeigt die Struktur des „data“-Ordners, der die Testdaten im CSV-Format enthält.

3.3.4 Python Konfiguration

Damit ein Python Projekt funktioniert muss es bestimmte Konventionen erfüllen. Dazu gehört, dass alle Verzeichnisse, die Python-Module enthalten sollen, eine `__init__.py`-Datei besitzen. Diese Dateien sind zwar leer, signalisieren dem Python-Interpreter jedoch, dass das Verzeichnis als Paket behandelt werden soll. In diesem Projekt werden sie in allen relevanten Verzeichnissen automatisch erstellt. Des weiteren wird eine `README.md`-Datei erstellt. Zum einen enthält sie Schritte zum Erstellen einer `vscode`-Konfigurationsdatei, `.vscode/settings.json`. Zum anderen enthält diese Schritte zum Einrichten einer virtuellen Umgebung namens „venv“. Dorthin werden alle Abhängigkeiten installiert, die in der `requirements.txt`-Datei aufgelistet sind, die auch vom Migrator erstellt wird. Die Abhängigkeiten stehen als Liste in der `requirements.txt`-Datei und fungieren als Referenz aller notwendigen Python-Pakete, die für die Ausführung der Tests erforderlich sind. Es wird eine `pytest.ini`-Datei erstellt, die die Konfiguration für Pytest enthält. Dort wird unter anderem der Pfad zu den Testskripten angegeben. Hier ist eine Darstellung dieser Konfigurationsdateien in Listing 9 zu sehen.

```
sample-website-selenium-tests/  
|  
├─ data/  
├─ src/  
├─ .gitignore  
├─ pytest.ini  
├─ README.md  
└─ requirements.txt
```

Listing 9: Übersicht über erstellten Konfigurationsdateien.

4 Konzeption der Migrationspipeline

4.1 Hintergrund

Als die Aufgabe der Migration des internen Testsystems auf ein anderes Ökosystem aufkam, war es ein erstes Konzept, ein Python Projekt zu schreiben, das Katalon-Tests direkt lesen und ausführen kann. Diese Idee wurde nach Rücksprache mit anderen Entwicklern jedoch schnell verworfen, da die proprietären Formate und die Katalon-spezifische Logik zu komplex waren, um sie direkt in einem Open-Source-Framework auszuführen. Stattdessen wurde ein Ansatz gewählt, der die Katalon-Testlogik komplett in eine neue, erweiterbare und frei nutzbare Struktur transformiert. Durch diese komplette Trennung von Katalon-Strukturen sollte sichergestellt werden, dass in der Zukunft keine weiteren Probleme mit Abhängigkeiten zu Katalon mehr aufkommen. Die Tests sollten in der neuen Struktur direkt ausgeführt werden können, ohne, dass Katalon Studio gebraucht würde falls beispielsweise neue Tests oder Elemente kreiert werden müssten. Als erster Ansatz wurde eine kleine Pipeline gebaut mit der Aufgabe, häufig vorkommende Methoden zu übersetzen. Dieser Ansatz zeigte sehr schnell, dass die Katalon Struktur im Hintergrund der IDE viel komplexer und verschachtelter war, als es auf den ersten Blick schien.

4.2 Architekturüberblick

Um die Komplexität zu bewältigen wird das Katalon Projekt strukturiert in seine Teile zerlegt. Ein *Vorteil* der proprietären Struktur ist es, dass sie die Struktur vorgibt und berechenbar ist, was diese Art von Algorithmus erst möglich macht. Zuerst bekommt der Algorithmus die Pfade des Quell- und des Zielprojekts vom User übergeben. Anschließend werden rekursiv alle Dateien im Quellprojekt durchsucht und die relevanten Strukturen, wie „Test Case“-Ordner und das Object Repository, werden im Zielprojekt als leere Ordner wiedergespiegelt. Die unveränderten Systempfade, die vom „root“-Verzeichnis zu den Dateien führen, sind dabei integral. Sind die Grundstrukturen somit initialisiert, werden Dateien aus der Quelle bearbeitet. Zuerst werden die Test Cases aus der Groovy Programmiersprache in die Python Sprache transpiliert. Den genaueren Ablauf davon kann man im Kapitel 4.3 nachlesen. Anschließend werden die Test Objects, die globalen Variablen sowie die Testdaten gefunden, gelesen und übersetzt. Alle von ihnen enthalten Extensible Markup Language (XML)-Daten, haben jedoch unterschiedliche Dateiendungen. Die Test Objects werden dabei zu JSON-Dateien umgewandelt, die Global Variables in Python-Klassen und die Testdaten werden unverändert übernommen. Die

testeigenen Variablen werden in einer Struktur gespeichert, die der der Test Cases exakt gleicht. Sie werden in Python-Klassen gespeichert und enthalten jeweils alle Variablen des entsprechenden Tests. Die Klassen werden dann in den Tests importiert. Zuletzt werden Konfigurationsdateien und Hilfsdateien im Zielprojekt erstellt, welches in Abbildung 6 zu sehen ist, damit dieses ohne größere Vorbereitungen ausführbar ist. Dazu zählen Laufzeithelfer, Testkonfigurationen, die Liste der Abhängigkeiten und Dokumentation.

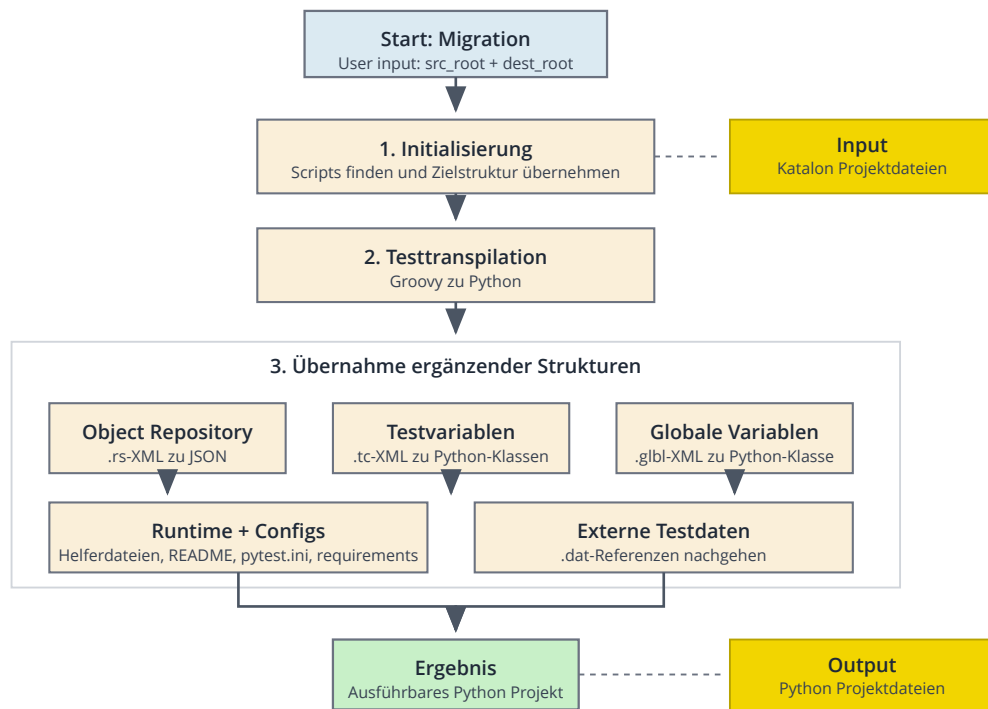


Abbildung 6: Übersichtsdiagramm der Migrationspipeline von einem Katalon Projekt zu einem ausführbaren Python Projekt. Dargestellt sind die Hauptphasen Initialisierung, Testtranspilation, Übernahme ergänzender Strukturen sowie die Bereitstellung der finalen Projektstruktur mit Inhalten.

4.3 Transpilation der Test Cases

Der in Abbildung 7 dargestellte Transpiliationsablauf folgt einer festen Abfolge von sieben Schritten. Zunächst wird die ursprüngliche Groovy-Datei eingelesen, damit ihr Inhalt als Roh-text verarbeitet werden kann. Anschließend werden Kommentare und Importzeilen entfernt, um den für die Migration relevanten Code zu isolieren. Im dritten Schritt werden daraus die eigentlichen Testzeilen extrahiert, also genau die Anweisungen, die für das Testverhalten maßgeblich sind. Diese Zeilen werden danach einzeln geparkt, sodass jede Anweisung in ihre grundlegenden Bestandteile wie Klassenbezug, Methodenname und Parameter zerlegt wird. Auf dieser strukturierten Grundlage folgt die Transformation der erkannten Methoden und Parameter in

inhaltlich entsprechende Konstrukte der Zielumgebung. Daraus wird anschließend Python-Code erzeugt, der die zuvor identifizierte Testlogik in ausführbarer Form abbildet. Im letzten Schritt werden die generierten Codefragmente zu einem vollständigen Test zusammengesetzt und in die Zielstruktur überführt. Der gesamte Ablauf reduziert den ursprünglichen Groovy-Test damit schrittweise auf seine wesentliche Logik und überführt sie systematisch in ein Python-basiertes Testformat.

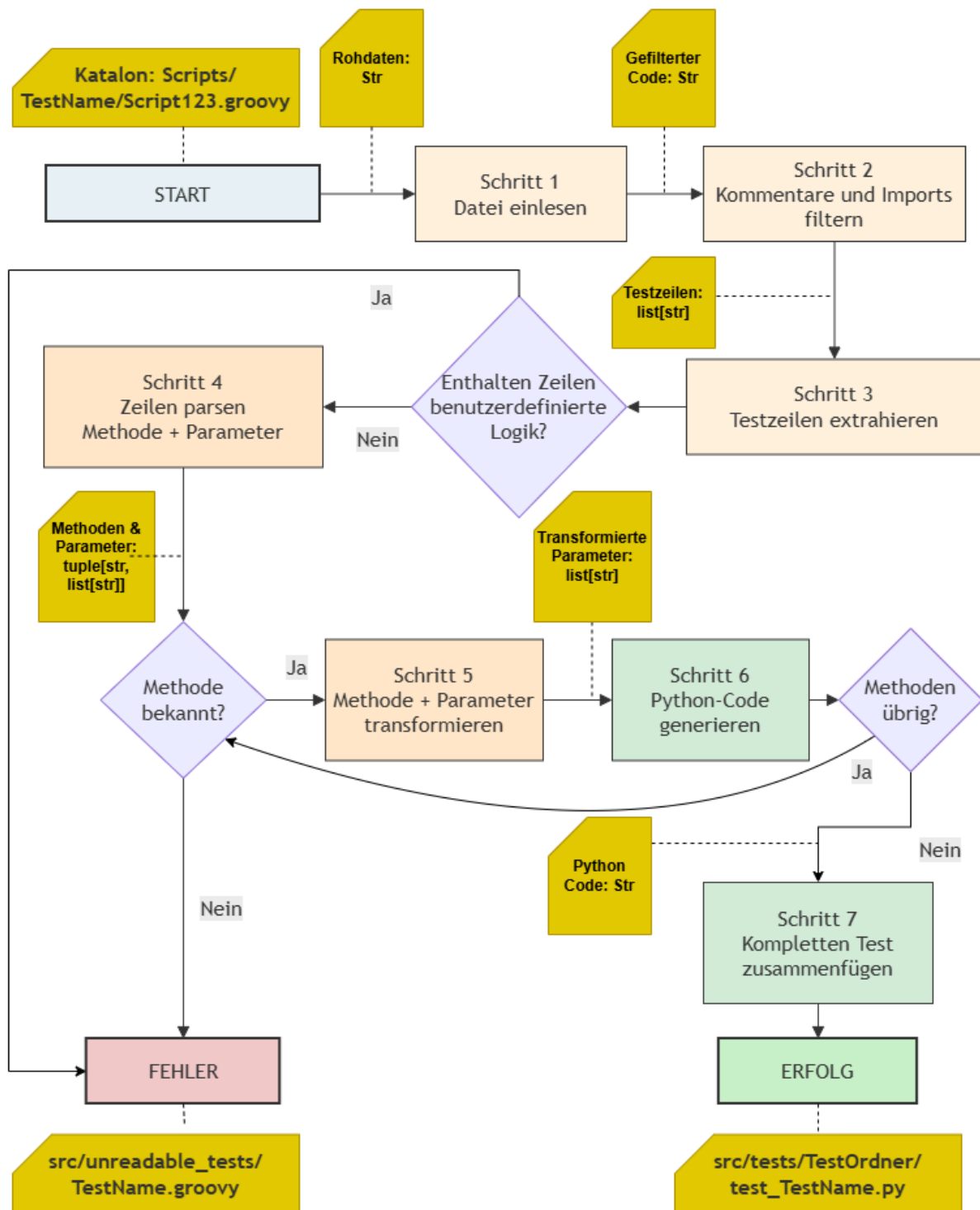


Abbildung 7: Detailliertes Aktivitätsdiagramm des Transpilationsablaufs eines Katalon-Tests zu einem Python-Test. Dargestellt sind das Einlesen, Filtern, Extrahieren, Parsing, Transformieren, Generieren und Zusammensetzen der Testlogik.

4.4 Übernahme ergänzender Strukturen

Ein ausführbares Zielprojekt entsteht erst dann, wenn neben der Testlogik auch die zugehörigen Kontextstrukturen existieren. Deswegen müssen nach der Transpilation der Test Cases auch die ergänzenden Strukturen übernommen werden. Dazu gehören insbesondere Objektbeschreibungen aus dem Object Repository, Variablenquellen - global oder lokal, Testdaten sowie Laufzeit- und Konfigurationsdateien. Ohne diese Elemente wären viele erzeugte Tests

zwar syntaktisch vorhanden, könnten aber nicht korrekt auf Selektoren, Daten oder Umgebungsparameter zugreifen.

Die Übernahme verfolgt deshalb das Ziel, die in Katalon verteilten Abhängigkeiten in eine offene und nachvollziehbare Zielstruktur zu überführen, ohne die semantische Bedeutung der Tests zu verlieren. Damit wird sichergestellt, dass nicht nur einzelner Code migriert wird, sondern ein konsistentes Testsystem entsteht, das im neuen Ökosystem unmittelbar weiterentwickelt und ausgeführt werden kann.

4.5 Limitierungen der Migrationspipeline

Die Migrationspipeline ist darauf ausgelegt, die Kernfunktionen eines Katalon Projektes zu erkennen und in eine neue Struktur zu transformieren. Sie ist jedoch nicht in der Lage, alle möglichen Katalon-Funktionen und -Szenarien abzudecken. Insbesondere eigens geschriebene Custom Keywords, plattformspezifische Erweiterungen oder stark verschachtelte Testlogiken können zu Problemen führen. Auch Kommentare werden wegen geringer Relevanz vorerst vorweggelassen. Die Pipeline konzentriert sich auf die häufigsten Anwendungsfälle und bietet eine solide Grundlage für die Migration, erfordert jedoch möglicherweise manuelle Anpassungen für spezielle Anforderungen. Das Schreiben von Regex-basierter Übersetzung dieser komplexen Strukturen ist sehr aufwendig und für den Rahmen der ursprünglichen Aufgabe wäre dies zu zeitintensiv gewesen.

5 Implementierung

Dieses Kapitel beschreibt die technische Umsetzung des Migrationswerkzeugs: seine Ordnerstruktur, die Trennung zwischen Migrations- und Laufzeitcode sowie die Aufgaben der einzelnen Pipeline-Module.

5.1 Techstack

5.1.1 Python

Die grundlegende Programmiersprache des Migrators ist Python. Die Wahl fiel auf Python, weil sich die Sprache durch eine vergleichsweise geringe Syntaxkomplexität, gute Lesbarkeit und eine breite Unterstützung für Dateiverarbeitung und Texttransformation auszeichnet. Diese Eigenschaften sind für einen Prototypen geeignet, der große Mengen strukturierter Eingabedaten analysieren, transformieren und in neue Quelltexte überführen soll.

5.1.2 Selenium und Pytest

Selenium ist ein etabliertes und weit verbreitetes Open-Source-Framework für Webbrowser-Automatisierung, das eine Vielzahl von Browsern und Plattformen unterstützt. Es bietet eine robuste API, die es ermöglicht, Webanwendungen zu steuern und zu testen. Pytest ist ein weiteres Open-Source-Framework, das für seine einfache Syntax, Flexibilität und umfangreiche Funktionalität bekannt ist. Es unterstützt die Erstellung von Testfällen, Test-Suites und bietet eine Vielzahl von Plugins zur Erweiterung der Funktionalität. Zusammen bilden sie einen vielseitigen und sehr praktikablen Techstack für die Testautomatisierung.

5.1.3 Git als Versionierung und GitHub

Zur Versionierung des Quellcodes wird Git verwendet. Das Versionskontrollsystem ermöglicht eine nachvollziehbare Entwicklung des Tools, da Änderungen historisiert, verglichen und bei Bedarf auf frühere Stände zurückgeführt werden können. Die Anbindung an GitHub erleichtert dazu die zentrale Ablage des Quellcodes und die dokumentierte Weiterentwicklung des Projekts.

5.2 Projektstruktur des Migrators

Die folgende Übersicht in Listing 10 zeigt die relevante Projektstruktur des Migrators auf Wurzelebene. Der Thesis-Ordner wird dabei bewusst ausgeblendet, damit die technische Arbeitsstruktur des Migrationstools im Fokus bleibt.

```
katalon-test-migrator/
├─ main.py
├─ README.md
├─ pyrightconfig.json
├─ .gitignore
└─ src/
    ├─ pipeline/
    │   ├─ test_script_scanner.py
    │   ├─ test_transpiler.py
    │   ├─ test_assembler.py
    │   ├─ object_repo_converter.py
    │   ├─ global_vars_generator.py
    │   ├─ variables_extractor.py
    │   ├─ copy_runtime_files.py
    │   └─ __init__.py
    ├─ runtime/
    │   ├─ base_test.py.template
    │   ├─ katalon_helpers.py.template
    │   └─ __init__.py
    └─ utils/
        ├─ file_utils.py
        ├─ string_utils.py
        └─ xml_utils.py
```

Listing 10: Projektstruktur des Migrationstools auf „root“-Ebene (ohne Thesis-Ordner & Continuous Integration (CI)-Pipeline) mit Trennung zwischen Steuerlogik, Konfigurationsdateien und Migrationsmodulen.

Die Struktur trennt Steuerlogik und Migrationsmodule klar voneinander. In der Projektwurzel orchestriert `main.py` den Gesamtablauf, während `README.md` die Ausführung dokumentiert. Im Ordner `src/` liegen die funktionalen Bausteine: `pipeline/` enthält die Transformationsschritte, `runtime/` die als Templates vorliegenden Laufzeithelfer für das Zielprojekt und `utils/` gemeinsam genutzte Hilfsfunktionen. Diese Aufteilung reduziert Kopplungen zwischen den Modulen und erleichtert die gezielte Erweiterung einzelner Pipeline-Bausteine.

5.3 Umsetzung der Migrationspipeline

Die Migrationspipeline besteht aus sieben spezialisierten Modulen. `main.py` ruft die steuernden Funktionen auf, die die Module in der Pipeline orchestrieren. Jedes Modul übernimmt einen klar abgegrenzten Teilschritt und überführt die Katalon-Struktur dadurch kontrolliert in die Zielumgebung.

Zur Robustheit der Pipeline werden fehlerhafte oder nicht eindeutig transformierbare Testskripte nicht verworfen, sondern separat in `src/unreadable_tests/` abgelegt. Dadurch bleibt der automatische Durchlauf stabil, während problematische Fälle gezielt manuell nachbearbeitet werden können.

`test_script_scanner.py` bildet den Einstiegspunkt der Pipeline. Das Modul durchläuft rekursiv den `Scripts/-`Ordner des Katalon-Projekts, filtert `.groovy`-Dateien und verteilt jede Datei an `test_transpiler.py` und `test_assembler.py`. Dabei wird die Katalon-Ordnerstruktur in `src/tests/` gespiegelt. Nicht vollständig übersetzbare Tests werden in `src/unreadable_tests/` abgelegt.

`test_transpiler.py` führt die syntaktische Transformation durch. Es wendet die in Tabelle 2 dokumentierten Regex Muster auf den gescannten Groovy-Code an, erkennt Katalon-Methodeaufrufe und überführt sie schrittweise in Python-Äquivalente. Das Ergebnis ist eine Liste transformierter Codezeilen.

`test_assembler.py` baut aus den transpilierten Zeilen eine vollständige Python-Pytest-Testklasse. Das Skript ergänzt die benötigten Imports, erzeugt eine Klassenstruktur mit `BaseTest`-Vererbung im Constructor und kapselt die Testlogik in einer `test_TESTNAME`-Methode.

`object_repo_converter.py` konvertiert, mit Hilfe von `xml_utils.py`, alle `.rs`-Dateien im `Object Repository/-`Ordner von XML nach JSON. Die interne `WebElementEntity`-Struktur bleibt erhalten; lediglich das Dateiformat ändert sich. Das Ergebnis wird unter `src/object_repository/` abgelegt.

`global_vars_generator.py` liest im Quellverzeichnis `Profiles/default.glbl` (XML mit `GlobalVariableEntity`-Einträgen) und erzeugt daraus `src/profiles/global_variables.py`, also eine Python-Klasse `GlobalVariables` mit Klassenvariablen für globale Projektwerte.

`variables_extractor.py` liest die `.tc`-Metadateien der Test Cases und extrahiert testeigene Variablen. Für jeden Test mit Variablen wird eine eigene Python-Datei unter `src/variables/` erzeugt, die die Variablen als Python-Klasse bereitstellt.

`copy_runtime_files.py` schließt die Migration ab. Dieses Skript kopiert `base_test.py.template` und `katalon_helpers.py.template` in `src/runtime/` des Zielprojekts entfernt die Endung `.template` und erzeugt die Konfigurationsdateien `pytest.ini`, `requirements.txt`, `.gitignore` und `README.md` in der Projektwurzel.

5.3.1 Transpilation der Test Cases

Die eigentliche Testtranspilation ist der Kern der Implementierung. Sie beginnt mit dem Einlesen der Groovy-Datei, entfernt nicht relevante Bestandteile wie Kommentare und Importzeilen und reduziert den Code anschließend auf die testrelevanten `webUI`-Zeilen. Diese Zeilen werden geparkt, in einzelne Bestandteile zerlegt und Schritt für Schritt in Python-Strukturen überführt. Dadurch entsteht aus der ursprünglichen Katalon-Logik ein ausführbarer Test, dessen Aufbau dem ursprünglichen Verhalten entspricht.

5.3.1.1 Semantische Transformation: Katalon Test Object zu Selenium Locators

Das Katalon Object Repository speichert Testobjekte als `.rs`-Dateien, die im Inneren umbenannte XML-Dateien mit einer `WebElementEntity`-Struktur sind. Jede Datei beschreibt ein HTML Element über eine `selectorCollection`, die eine oder mehrere Lokalisierungsstrategien als Key-Value-Paare enthält (z. B. BASIC/XPath oder CSS), sowie ein `selectorMethod`-Feld, das die bevorzugte Strategie bestimmt.

Während der Build-Time konvertiert `object_repo_converter.py` diese XML-Dateien in das JSON-Format. Die interne Struktur bleibt dabei vollständig erhalten und nur das Dateiformat ändert sich. Das folgende Beispiel in Listing 11 zeigt die Transformation der Objektdatei `view_all_users_btn`:

```
<!-- Object Repository/All_Users/view_all_users_btn.rs (Katalon XML) -->
<WebElementEntity>
  <name>view_all_users_btn</name>
  <selectorCollection>
    <entry>
      <key>BASIC</key>
      <value>//*[@id = 'hero-cta']</value>
    </entry>
  </selectorCollection>
  <selectorMethod>BASIC</selectorMethod>
</WebElementEntity>

// src/object_repository/All_Users/view_all_users_btn.json (generiertes JSON)
{
  "WebElementEntity": {
    "name": "view_all_users_btn",
    "selectorCollection": {
      "entry": { "key": "BASIC", "value": "//*[@id = 'hero-cta']" }
    },
    "selectorMethod": "BASIC"
  }
}
```

Listing 11: Beispielhafte Transformation eines Katalon-Testobjekts von einer XML-basierenden `.rs`-Datei in die entsprechende JSON-Darstellung im Zielprojekt.

Zur Runtime im Zielprojekt liest die Hilfsfunktion `find_katalon_test_object()` in `katalon_helpers.py` die JSON-Datei ein, liest `selectorMethod` aus und sucht den passenden Wert aus der `selectorCollection`. Anschließend lokalisiert sie das Element über die API von Selenium. Im Fall von CSS mit `By.CSS_SELECTOR`, bei allen anderen Strategien, einschließlich BASIC, mit `By.XPATH`. Der Aufruf im generierten Testcode bleibt dabei strukturell identisch zum Katalon-Original, zu sehen in Listing 12:

```
// Katalon (Groovy):
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
```

```
# Generierter Python-Code:
```

```
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
```

Listing 12: Beispielhafte Abbildung eines Katalon-`findTestObject()`-Aufrufs auf den entsprechenden Selenium-basierten Laufzeithelfer im generierten Python-Test.

Der generierte Testcode hat selbst keine Kenntnis über die verwendete Lokalisierungsstrategie, sondern greift nur auf den Laufzeithelfer `find_katalon_test_object()` zu und übergibt einen Pfad. Die Bindung zwischen Testlogik und Element-Selektor bleibt erhalten, ohne, dass sich die Testeingabe ändert.

5.3.1.2 Syntaktische Transformation: Groovy zu Python

Die Groovy-Syntax wird durch Regex-gestützte Mustererkennung schrittweise in Python-Syntax überführt. Dabei werden die ursprünglichen Katalon-Aufrufe nicht nur syntaktisch ersetzt, sondern in eine Form überführt, die innerhalb der generierten Pytest-Struktur unmittelbar ausführbar ist. Listing 13 zeigt beispielhaft, wie typische WebUI-Aufrufe, Objektzugriffe und Variablenreferenzen transformiert werden.

```
WebUI.verifyTextPresent('View All Users', false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), david)
WebUI.verifyTextPresent(GlobalVariable.user4name, false)

assert 'View All Users' in self.driver.page_source
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
kh.find_katalon_test_object(self.driver, 'All_Users/search_input').clear()
kh.find_katalon_test_object(self.driver, 'All_Users/
search_input').send_keys(variables.david)
assert GlobalVariable.USER4NAME in self.driver.page_source
```

Listing 13: Beispielhafte syntaktische Transformation mehrerer Katalon-WebUI-Aufrufe in ausführbaren Python-Testcode innerhalb der generierten Pytest-Struktur.

Das Beispiel zeigt drei typische Transformationsarten: Assertions auf Seiteninhalte werden in Python-Assertions überführt, `findTestObject()`-Aufrufe werden mithilfe der Laufzeithelfer zur Lokalisierung der Zielobjekte abgebildet, und Variablenzugriffe werden an die im Zielprojekt erzeugten Python-Strukturen angebunden. Besonders relevant ist dabei, dass Pfadangaben und Objektbezeichner nicht verworfen, sondern in ein neues Zugriffsschema übertragen wer-

den. Dadurch bleibt die Bindung zwischen Testlogik und ergänzenden Strukturen auch nach der Migration erhalten.

5.3.1.2.1 Regex-basierte Transformation

Die Pipeline nutzt mehrere aufeinander abgestimmte Muster, um Groovy-Testcode schrittweise in ein ausführbares Python-Testformat zu überführen.

Im Sinne des Diagramms beginnt dieser Ablauf mit dem Einlesen der ursprünglichen Groovy-Datei. Danach werden Kommentare und Importzeilen entfernt, sodass nur der für die Transpilation relevante Code verbleibt. Aus diesem gefilterten Code werden im nächsten Schritt die eigentlichen Testzeilen extrahiert. Diese Testzeilen werden anschließend geparkt und in ihre Grundbestandteile wie Klassenbezug, Methodenname und Parameter zerlegt. Auf dieser Grundlage folgt die Transformation erkannter Methoden und Parameter in entsprechende Konstrukte der Zielumgebung. Aus den transformierten Bausteinen wird danach Python-Code erzeugt, der die ursprüngliche Testlogik in neuer Form abbildet. Abschließend werden die generierten Fragmente zu einem vollständigen Test zusammengesetzt.

Die folgenden Abbildungen veranschaulichen exemplarisch, wie einzelne Muster im Parsing eingesetzt werden. Gezeigt wird insbesondere, wie relevante Testzeilen zunächst isoliert und anschließend `findTestObject()`-Aufrufe als eigenständige Parameterbestandteile erkannt werden. Dadurch wird sichtbar, dass die Regex-Muster nicht unabhängig voneinander arbeiten, sondern eine aufeinander aufbauende Verarbeitungskette bilden. Die Beispiele in Abbildung 8, Abbildung 9 und Abbildung 10 zeigen die Muster in Aktion und verdeutlichen, wie sie die Katalon-Testlogik Schritt für Schritt in eine strukturierte Form überführen, die für die spätere Transformation in Python-Code genutzt werden kann.

Regular Expression
Matches · 3

: / (\w+)\.(\w+)\((.*)\)\

Test String

```
WebUI.verifyTextPresent('View*All*Users', *false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), *david)
```

Explanation

- 1st Capturing Group (\w+)
 - \w+ matches any word character (equivalent to [a-zA-Z0-9_])
 - + matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)
- \. matches the character . with numeric value 46₁₀ (56₈ or 2E₁₆) literally
- 2nd Capturing Group (\w+)
 - \w+ matches any word character (equivalent to [a-zA-Z0-9_])
 - + matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)
- \(matches the character (with numeric value 40₁₀ (50₈ or 28₁₆) literally

Match Information

Match 1	0-48	WebUI.verifyTextPresent('View*All*Users', *false)
Group 1	0-5	WebUI
Group 2	6-23	verifyTextPresent
Group 3	24-47	'View*All*Users', *false
Match 2	50-109	WebUI.click(findTestObject('All_Users/view_all_users_btn'))
Group 1	50-55	WebUI
Group 2	56-61	click
Group 3	62-108	findTestObject('All_Users/view_all_users_btn')
Match 3	111-173	WebUI.setText(findTestObject('All_Users/search_input'), *david)

Abbildung 8: Durch das katalon_lines_pattern-Muster werden aus einem Block aus Katalon Zeilen, drei getrennte Regex-Matches mit jeweils drei Gruppen für Klasse, Methode und Parameter extrahiert (Visualisierung und Validierung mit Regex101).

Regular Expression
Matches · 1
Steps

: / (findTestObject\(.*\)\)(?=',)

Test String

```

WebUI.verifyTextPresent('View-All-Users', *false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), *david)

```

Explanation

- 1st Capturing Group (findTestObject\(.*\))
 - findTestObject matches the characters findTestObject literally (case sensitive)
 - \ (matches the character (with numeric value 40₁₀ (50₈ or 28₁₆) literally
 - .* matches any character (except for line terminators)
 - * matches the previous token zero or more times, as many times as possible, giving characters back as needed (greedy)
 - \) matches the character) with numeric value 41₁₀ (51₈ or 29₁₆) literally
- Positive Lookahead (?=',)
 - , matches the character , with numeric value 44₁₀ (54₈ or 2C₁₆) literally

Match Information

Match 1	125-165	findTestObject('All_Users/search_input')
Group 1	125-165	findTestObject('All_Users/search_input')

Abbildung 9: Das `fto_as_param_pat`-Muster erkennt in der Parsing die `findTestObject()`-Methode als ersten Parameter, falls ein Komma folgt und ermöglicht es diese zu trennen (Visualisierung und Validierung mit Regex101).

Regular Expression
Matches

```

: / (findTestObject\(((?=.*+').*\))

```

Test String

```

WebUI.verifyTextPresent('View-All-Users', 'false')
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), 'david')

```

Explanation

- 1st Capturing Group `(findTestObject\(((?=.*+').*\))`
 - `findTestObject` matches the characters `findTestObject` literally (case sensitive)
 - `\(` matches the character `(` with numeric value `4010` (`508` or `2816`) literally
 - 2nd Capturing Group `((?=.*+').*\))`
 - `'` matches the character `'` with numeric value `3910` (`478` or `2716`) literally
 - `.*+` matches any character (except for line terminators)
 - `+` matches the previous token one or more times, as many times as possible, giving characters back as needed (greedy)
 - `'` matches the character `'` with numeric value `3910` (`478` or `2716`) literally

Match Information

Match 1	62-109	<code>findTestObject('All_Users/view_all_users_btn')</code>
Group 1	62-109	<code>findTestObject('All_Users/view_all_users_btn')</code>
Group 2	77-107	<code>'All_Users/view_all_users_btn'</code>
Match 2	125-173	<code>findTestObject('All_Users/search_input'), 'david'</code>
Group 1	125-173	<code>findTestObject('All_Users/search_input'), 'david'</code>
Group 2	140-164	<code>'All_Users/search_input'</code>

Abbildung 10: Das `fto_param_str_pattern`-Muster extrahiert die `findTestObject()`-Methode mit dem String-Argument, das für die Transformation benötigt wird (Visualisierung und Validierung mit Regexp101).

Eine vollständige Übersicht der verwendeten Regex Muster befindet sich im Anhang in Kapitel 8.1. Dort sind die wichtigsten Muster, ihre regulären Ausdrücke und ihr jeweiliger Zweck zusammengefasst.

5.4 Grenzen der Implementierung

Die implementierte Pipeline ist auf die im Projekt beobachteten Standardfälle ausgerichtet. Sie verarbeitet die typischen Katalon-Strukturen zuverlässig, stößt aber bei von der Norm abweichenden Groovy-Konstrukten, ungewöhnlich verschachtelten Testabläufen oder projektindividuellen Custom Keywords an Grenzen. Solche Fälle erfordern entweder ergänzende Regeln oder eine manuelle Nacharbeit im Zielprojekt. Damit bleibt die Implementierung bewusst fokussiert auf die häufigsten Migrationspfade, anstatt jede theoretisch mögliche Katalon-Struktur vollständig abzubilden.

Nach der Darstellung der Implementierungsdetails wird im folgenden Kapitel überprüft, in welchem Umfang die Pipeline die erwarteten Migrationsziele im Beispielprojekt tatsächlich erreicht.

6 Evaluation

Zur Evaluation des Migrationswerkzeugs wurde das Katalon-Beispielprojekt `sample-website-katalon-tests` verwendet. Es enthält reale End-to-End-Tests, ein Object Repository, zwei globale Variablen-Profile und eine eingebundene CSV-Datendatei. Die Migration wurde auf einem Windows-11-Rechner mit Python 3.13.0 und pytest 8.4.1 durchgeführt.

6.1 Evaluationskriterien

Die Bewertung des Prototyps orientiert sich bewusst an den Basisfunktionen der Migrationspipeline und den in dieser Arbeit formulierten Kernzielen.

Im Migrationsergebnis wird beobachtet, welcher Anteil der relevanten Katalon-Dateien erfolgreich in die Zielstruktur überführt wurde. Zudem wird auch der Automatisierungsgrad bewertet. Dabei wird erfasst, welche Transformationsschritte ohne manuelle Eingriffe durchlaufen und welche Artefakte im automatischen Durchlauf vollständig erzeugt werden.

Bei der Prüfung der strukturellen Integrität wird untersucht, ob das erzeugte Python-Projekt die ursprüngliche Testlogik in vergleichbarer Weise ausführt und ob die erwarteten Testfälle korrekt bereitgestellt werden.

Anschließend wird die Codequalität bemessen. Dazu zählen der komplette Syntax, nicht oder nur teilweise übersetzte Fälle und deren Ursachen.

Als letztes wird der Aufwand zwischen manuellem Migrieren und der Nutzung der Migrationspipeline verglichen.

6.2 Migrationsergebnis

Die Ausführung des Migrators (`python main.py`) liefert das in Tabelle 1 verdeutlichte Ergebnis:

Tabelle 1: Migrationsergebnis

Komponente	Anzahl	Beschreibung
Übersetzte Testskripte	7	7 von 10 .groovy-Dateien aus Scripts/ erfolgreich übersetzt
Nicht übersetzbare Tests	3	3 Tests konnten nicht automatisch übersetzt werden
Object Repository Dateien	10	Alle .rs XML-Dateien zu JSON konvertiert
Variablen-Dateien	1	Testeigene Variablen aus .tc-Metadateien extrahiert
Runtime-Dateien	2	base_test.py und katalon_helpers.py ins Ziel kopiert
Konfigurations-Dateien	4	pytest.ini, requirements.txt, .gitignore, README.md
Datendateien	1	users.csv direkt ins Ziel-Projekt kopiert

Von 10 vorhandenen Testskripten wurden 7 automatisch übersetzt, während 3 Fälle fehlschlugen. Damit liegt die Übersetzungsabdeckung für Testskripte bei **70%**, mit einem unübersetzten Rest von **30%**. Abbildung 11 zeigt einen Konsolenausschnitt der Migration.

```

Migration started
[verifyEqual] is an unknown testmethod
[waitForElementVisible] is an unknown testmethod
[verifyElementVisible] is an unknown testmethod
[verifyElementVisible] is an unknown testmethod
[verifyElementVisible] is an unknown testmethod
[waitForElementVisible] is an unknown testmethod
Tests: 7 translated, 3 unreadable, 3 folders prepared
Object repository: 10 files converted
Variables: 1 files generated, 0 folders created
Runtime: 2 files copied, 3 root config files created, 0 existing .gitignore merged, 1 existing .gitignore unchanged
Data files: 1 copied, 0 warnings
Migration finished

```

Abbildung 11: Konsolenausschnitt der Migration mit 7 erfolgreichen und 3 fehlgeschlagenen Testübersetzungen.

6.3 Strukturelle Integrität

Hier wird geprüft ob das Zielprojekt vollständig importierbar ist und die erwarteten Testfälle korrekt bereitgestellt werden.

Nach der Migration wurde das Python-Projekt mit `pytest --collect-only` geprüft. Das ist in Listing 14 verdeutlicht. Eine Pytest-native Funktion, die alle Tests durchsucht und den Syntax kontrolliert. Pytest konnte alle generierten Dateien fehlerfrei importieren und die 7 automatisch erzeugten Tests entdecken.

```
collected 7 items
```

```
src/tests/Menus/test_navigation_bar.py::Test_navigation_bar::test_navigation_bar
src/tests/Users/test_filter_for_admins.py::Test_filter_for_admins::test_filter_for_admins
src/tests/Users/
test_search_for_david_kim.py::Test_search_for_david_kim::test_search_for_david_kim
src/tests/Users/test_show_active_viewers.py::Test_show_active_viewers::test_show_active_viewers
src/tests/Users/test_show_editors.py::Test_show_editors::test_show_editors
src/tests/Users/
test_verify_users_controls_visible.py::Test_verify_users_controls_visible::test_verify_users_controls_visible
src/tests/Util/test_login.py::Test_login::test_login
```

```
===== 7 tests collected in 0.46s =====
```

Listing 14: Konsolenausgabe von `pytest --collect-only`, die zeigt, dass alle 7 automatisch erzeugten Python-Tests fehlerfrei erkannt werden.

Die folgenden Screenshots in Abbildung 12 zeigen die Testausführung aller Tests in Katalon Studio, mit 10 von 10 Erfolgen und die Testausführung im generierten Python-Projekt, mit 7 von 7 Erfolgen.

(10/10) test cases ✓ 10 Passed ✗ 0 Failed ! 0 Error ⏸ 0 Incomplete ▶ 0 Skipped

✓	Test Suites/All_Tests	51.349s
✓	Test Cases/Menus/navigation_bar	6.770s
✓	Test Cases/Menus/show_amount_of_entries	3.417s
✓	Test Cases/Users/filter_for_admins	5.107s
✓	Test Cases/Users/search_for_david_kim	7.820s
✓	Test Cases/Users/show_active_viewers	4.218s
✓	Test Cases/Users/verify_user_count_more_than_five	3.215s
✓	Test Cases/Users/verify_users_controls_visible	4.405s
✓	Test Cases/Users/verify_combined_filters_return_results	4.243s
✓	Test Cases/Users/show_editors	5.117s
✓	Test Cases/Util/login	1.326s

```
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0
rootdir: c:\Repos\Bachelor Repos\sample-website-selenium-tests
configfile: pytest.ini
collected 7 items

src\tests\Menus\test_navigation_bar.py .           [ 14%]
src\tests\Users\test_filter_for_admins.py .         [ 28%]
src\tests\Users\test_search_for_david_kim.py .      [ 42%]
src\tests\Users\test_show_active_viewers.py .       [ 57%]
src\tests\Users\test_show_editors.py .              [ 71%]
src\tests\Users\test_verify_users_controls_visible.py . [ 85%]
src\tests\Util\test_login.py .                     [100%]

===== 7 passed in 31.40s =====
Finished running tests!
```

Abbildung 12: Vergleich der Testausführung: oben alle Tests im Katalon-Ausgangsprojekt, unten die Ausführung im generierten Python-Projekt. Die Abweichung entspricht den 3 nicht automatisch übersetzten Fällen.

6.4 Codequalität: Vorher-Nachher-Vergleich

Am Beispiel des Tests `filter_for_admins`, in Abbildung 13, lässt sich der Unterschied zwischen ursprünglichem Katalon-Skript und generiertem Python-Test visuell nachvollziehen:



Abbildung 13: Vergleich des Tests `filter_for_admins`: oben das ursprüngliche Katalon-Skript in Groovy, unten der daraus generierte Python-Test im Zielprojekt.

Der Boilerplate-Anteil sinkt von 18 auf 7 Importzeilen. Die Testlogik ist in standardmäßigem Python mit pytest-Konventionen formuliert und benötigt keine proprietären Katalon-Keywords mehr.

Im derzeitigen Stand der Pipeline werden alle Kommentare des ursprünglichen Codes aus Gründen der Komplexität entfernt.

6.5 Aufwandsvergleich

Die automatisierte Migration des Beispielprojekts dauerte unter einer Sekunde und überführte 7 von 10 Testskripten direkt in die Zielstruktur. Für die verbleibenden 3 fehlgeschlagenen Fälle

ist eine manuelle Nacharbeit erforderlich. Eine vollständig manuelle Migration hätte dagegen zunächst die in Kapitel 3.2 beschriebenen proprietären Verschachtelungen zwischen Skripten, Test Cases, Objektdefinitionen, Profilen und Testdaten auflösen müssen, bevor anschließend 10 Groovy-Testskripte, 10 XML-basierte Objektdateien sowie die zugehörigen Variablen- und Konfigurationsstrukturen einzeln in Python überführt werden könnten. Dass ein solcher manueller Aufwand mit wachsender Komplexität schnell ansteigt, deckt sich mit der in der Testliteratur beschriebenen Beobachtung, dass manuelle Prüf- und Transformationsaufgaben bei steigender Fallzahl und Kombinationsvielfalt stark skalieren[4, S. 34, S. 37].

Für das hier betrachtete Beispielprojekt ist deshalb selbst bei konservativer Projektschätzung von einem manuellen Aufwand im hohen einstelligen bis zweistelligen Stundenbereich auszugehen. Die Teilautomatisierung reduziert diesen Aufwand deutlich, lässt aber für die 3 nicht übersetzten Testfälle weiterhin gezielte Transformations- und Prüfaufgaben offen. Mit wachsender Testanzahl skaliert der Migrator im Wesentlichen linear, während der manuelle Aufwand durch zusätzliche Querbezüge zwischen Variablen, Objekten und Datenstrukturen überproportional zunimmt.

7 Fazit und Ausblick

Die Ausführung des Prototyps ergibt ein Python-Projekt, das den Großteil der eingeführten Testlogik samt umgebender Testartefakte enthält. Im Beispielprojekt werden 70% der Testskripte sowie alle dazugehörigen Strukturen wie Test Objects, Testdaten und Variablen fehlerfrei automatisch transformiert. Das erzeugte Projekt ist nach minimalem Setup direkt ausführbar. Die restlichen 30% müssen jedoch manuell nachbearbeitet werden, um vollständig nutzbar zu sein.

In der Evaluation werden insbesondere die strukturelle und syntaktische Integrität überprüft. Demnach ist das Migrationstool kein vollständiger Ersatz für eine manuelle Migration, aber eine wirksame Grundlage zur deutlichen Reduktion des Aufwands.

Insgesamt zeigt die Arbeit, dass die Migration von Katalon-Projekten in eine Python-Testumgebung mit Selenium und Pytest machbar ist, auch wenn nicht alle Fälle automatisch abgedeckt werden können.

Für die Weiterentwicklung des Tools ergeben sich drei zentrale Richtungen. Erstens kann die Abdeckung erhöht werden, indem weitere Katalon Studio-interne Keyword-Methoden in die Liste der übersetzbaren Methoden aufgenommen werden. Zweitens können die Regex-Muster und Parsing-Regeln erweitert werden, um Kommentare, tiefere Verschachtelungen in Parameterlisten sowie eigens erstellte Variablen, Methoden und Klassen robuster zu verarbeiten. Auch Programmierschleifen und weitere Code-Konstrukte könnten in die Übersetzung aufgenommen werden. Dazu kann die Fehlerklassifikation und -behandlung verbessert werden, etwa durch eine Kategorisierung nicht übersetzter Skripte und eine detailliertere Fehlerausgabe, um die manuelle Nacharbeit gezielter und schneller durchführen zu können.

8 Anhang

8.1 Regex-Muster der Transpilation

Tabelle 2: Regex-Muster die in der Transpilation verwendet werden.

#	Pattern Name	Regex	Zweck
1	comment_pattern	/*[\\^]**(?:[\\^/*][\\^]**+)**/	Block-Kommentare entfernen
2	private_method_pat	private void .*{\\n[\\s\\w.\\(\\)=\\\"\\', '\\\\[+@\\-\\];\\<{}}*\\n}	Private Methoden extrahieren
3	katalon_lines_pattern	\\/* \\/*\\.+ \\/*.*+ WebUI.+\\n.+ WebUI.+ CustomKeywords.*	Relevante Zeilen filtern
4	katalon_code_pattern	(\\w+)\\. (\\w+)\\(((.*)\\)	Teilt Code in Klasse, Methode und Parameter
5	fto_as_param_pat	(findTestObject\\(\\.(.*)\\))(?=,)	Erkennt findTestObject() als ersten Parameter mit Lookahead auf folgendes Komma
6	ftd_as_param_pat	(findTestData\\(\\.(.*)\\))(?=,)	Erkennt findTestData() als ersten Parameter mit Lookahead auf folgendes Komma
7	param_pattern	,\\s+(?=false) (?!\\s),\\s(?!=Fail.*) (?![a-zA-Z]),\\s(?!\\s)(?![a-zA-Z]) ,\\s(?!null) ,\\s+(?!=\\[)	Parameter teilen, wenn mehrere vorhanden
8	fto_param_str_pattern	(findTestObject\\(\\('.*').*\\))	Extrahiert findTestObject() mit String-Argument zur Transformation
9	ftd_param_str_pattern	findTestData\\(\\('.*').*\\)\\.getValue\\(\\+\\)\\)	Extrahiert findTestData() mit String und getValue()-Argument
10	GlobalVariable pattern	GlobalVariable\\.([A-Za-z_][A-Za-z0-9_]*)	Global Variablen normalisieren

11	abn_test_pat	String\s\w+\s= if\(TestObject\s\w+\s=	Custom Code erkennen
----	--------------	--	----------------------

8.2 Katalon Projektstruktur

```

sample-website-katalon-tests/
|
├─ Test Cases/                                - Katalon IDE zeigt diese Ansicht
|   └─ Users/
|       └─ filter_for_admins.tc                - .tc = XML Meta-Datei (Beschreibung, Variable)
|       └─ search_for_david_kim.tc            - KEINE Testlogik enthalten!
|       └─ show_active_viewers.tc
|   └─ Menus/
|       └─ navigation_bar.tc
|
├─ Scripts/                                    - Echte Testlogik (außer IDE verborgen)
|   └─ Users/
|       └─ filter_for_admins/
|           └─ Script1781348815820.groovy      - Zufällige Nummer! (!)
|       └─ search_for_david_kim/
|           └─ Script1781447000826.groovy      - Entspricht search_for_david_kim.tc
|       └─ show_active_viewers/
|           └─ Script...groovy
|   └─ Menus/
|       └─ navigation_bar/
|           └─ Script...groovy
|
├─ Object Repository/                          - Lokalisierungsstrategien (Selektoren)
|   └─ All_Users/
|       └─ view_all_users_btn.rs                - .rs = XML Objekt (XPath, CSS, ID...)
|       └─ search_input.rs
|       └─ roles_dropdown.rs
|       └─ status_dropdown.rs
|       └─ user_row.rs
|       └─ user_row_view_btn.rs
|   └─ Nav_Bar/
|       └─ ...weitere Test Objects
|
├─ Profiles/                                    - Globale Variablen nach Umgebung
|   └─ default.glbl                             - .glbl = XML (GlobalVariableEntity Einträge)
|
├─ Data Files/                                  - Externe Testdaten-Referenzen
|   └─ users.dat                                - .dat = XML Meta (verweist auf echte Datei)
|
├─ Keywords/                                    - Custom Methoden/Helper
|   └─ data/
|       └─ TESTER.groovy                        - Groovy Klasse mit eigenen Methoden
|
├─ Libs/
|   └─ CustomKeywords.groovy                    - Statische Weiterleitungen zu Keywords
|
└─ Plugins/, Test Listeners/, ...

```

Quellenverzeichnis

- [1] regex101, „regex101: build, test, and debug regular expressions“. Zugegriffen: 31. Juli 2026. [Online]. Verfügbar unter: <https://regex101.com/>
- [2] Katalon, Inc., „Katalon | The AI Platform for Software Quality“. Zugegriffen: 21. Juli 2026. [Online]. Verfügbar unter: <https://katalon.com/>
- [3] International Software Testing Qualifications Board, „ISTQB Glossary“. Zugegriffen: 22. Juli 2026. [Online]. Verfügbar unter: <https://glossary.istqb.org/>
- [4] D. R. Kuhn, R. N. Kacker, und Y. Lei, „Practical Combinatorial Testing“, NIST Special Publication 800–142, 2010. doi: 10.6028/NIST.SP.800-142.
- [5] S. Chittala, „Enhancing Developer Productivity Through Automated CI/CD Pipelines: A Comprehensive Analysis“, *International Journal of Computer Engineering and Technology (IJCET)*, Bd. 15, Nr. 5, S. 882–891, 2024, doi: 10.5281/zenodo.13929524.
- [6] Selenium Contributors, „WebDriver | Selenium“. Zugegriffen: 30. Juli 2026. [Online]. Verfügbar unter: <https://www.selenium.dev/documentation/webdriver/>
- [7] A. Sahay, A. Indamutsa, D. Di Ruscio, und A. Pierantonio, „Supporting the Understanding and Comparison of Low-Code Development Platforms“, in *Proceedings of the 46th Euro-micro Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2020, S. 171–178. doi: 10.1109/SEAA51224.2020.00036.
- [8] C. Shapiro und H. R. Varian, *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business School Press, 1998.
- [9] J. E. Friedl, *Mastering Regular Expressions*, 3rd Aufl. Sebastopol, CA: O'Reilly Media, 2006.
- [10] devalx88, „Katalon Studio 8 with Eclipse RCP 4.13 - Miscellaneous - Katalon Community“. Zugegriffen: 31. Juli 2026. [Online]. Verfügbar unter: <https://forum.katalon.com/t/katalon-studio-8-with-eclipse-rcp-4-13/36808>
- [11] Katalon Docs, „Spy Web utility in Katalon Studio | Katalon Docs“. Zugegriffen: 31. Juli 2026. [Online]. Verfügbar unter: <https://docs.katalon.com/katalon-studio/record-and-spy/webui-record-and-spy-utilities/spy-web-utility-in-katalon-studio>
- [12] pytest development team, „Pytest Documentation: Configuration“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/customize.html>
- [13] pytest development team, „Pytest Documentation: Good Integration Practices“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/explanation/goodpractices.html>

- [14] Python Packaging Authority, „Python Packaging User Guide: src layout vs flat layout“. Zugriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/>

Erklärung zur selbstständigen Bearbeitung

Ich erkläre hiermit, dass ich die vorliegende wissenschaftliche Arbeit selbständig und ohne unerlaubte Hilfe angefertigt habe, ich andere als die angegebenen Quellen und Hilfsmittel nicht benutzt habe, ich die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe, die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfbehörde vorgelegen hat.

Ort

Datum

Unterschrift im Original